

Trust Region Methods

Theory, Convergence, and Applications

ISE 5406: Optimization II

Virginia Tech

Spring 2026

Outline

- 1 Motivation and Philosophy
- 2 The Trust Region Subproblem
- 3 Solving the Subproblem Approximately
- 4 The Trust Region Algorithm
- 5 Convergence Theory
- 6 Local Convergence Rates
- 7 Worst-Case Complexity
- 8 Applications
- 9 Case Study: Trust Regions for Quadrotor Estimation
- 10 Case Study: Stewart Platform Trajectory Planning
- 11 Detailed Applications
- 12 Trust Regions in the Wild: High-Impact Applications
- 13 Summary

Line Search vs. Trust Region: Two Philosophies

Line Search

- 1 Choose a direction p_k
- 2 Find a step length $\alpha_k > 0$
- 3 Update: $x_{k+1} = x_k + \alpha_k p_k$

Philosophy: Choose direction first, then decide *how far* to go.

Trust Region

- 1 Choose a region size $\Delta_k > 0$
- 2 Find a step p_k within the region
- 3 Update: $x_{k+1} = x_k + p_k$

Philosophy: Decide *how far* to trust the model, then choose direction and length simultaneously.

Trust regions couple the direction and step length decisions.

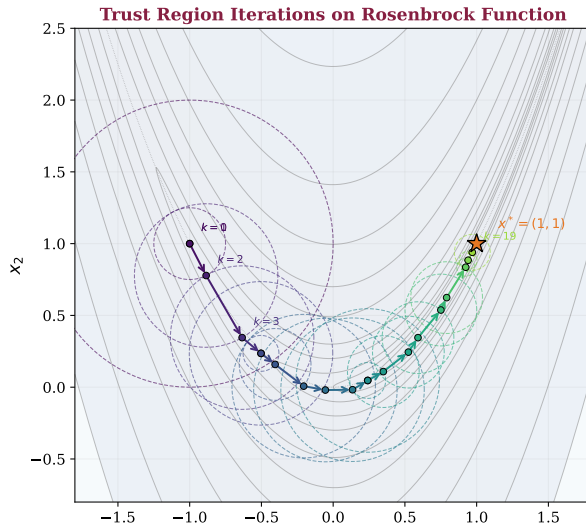
Why Trust Regions?

- **Robustness:** Handle indefinite Hessians naturally (no need for Hessian modification as in line search Newton).
- **Global convergence:** Strong theoretical guarantees under mild assumptions.
- **Adaptive step control:** The radius Δ_k adapts to local model quality.
- **Natural for constrained problems:** The trust region itself is a constraint — connects to augmented Lagrangian and SQP methods.

Historical Note

The trust region idea traces back to Levenberg (1944) and Marquardt (1963) for nonlinear least-squares problems. The general framework was formalized by Powell (1970s) and refined by Moré, Sorensen, and others.

Preview: Trust Regions in Action



Where trust regions appear:

- **Machine learning:** TRPO/PPO — the algorithms behind RLHF for ChatGPT, Claude, Gemini
- **Computer vision:** Bundle adjustment in Google Maps, SLAM, autonomous driving
- **Robotics:** Inverse kinematics via Levenberg-Marquardt
- **Power systems:** Optimal power flow for grid management
- **Medicine:** Radiation therapy dose optimization

This lecture: The theory behind all of these — N&W Chapter 4.

The Quadratic Model

At iterate x_k , we approximate f by the quadratic model:

$$m_k(p) = f(x_k) + \nabla f(x_k)^T p + \frac{1}{2} p^T B_k p$$

where B_k is either $\nabla^2 f(x_k)$ or an approximation (e.g., BFGS, SR1).

We write $g_k = \nabla f(x_k)$ for brevity.

Trust Region Subproblem (TRS)

$$\min_{p \in \mathbb{R}^n} m_k(p) \quad \text{subject to} \quad \|p\| \leq \Delta_k$$

Key features:

- The norm $\|\cdot\|$ is typically the ℓ_2 norm, but can be any norm.
- Even when B_k is indefinite, the TRS has a solution (compact feasible set, continuous objective $\Rightarrow$ Weierstrass).

Motivating Example: Setting Up KKT

Before stating the general theorem, let's **derive** the optimality conditions ourselves via KKT on a concrete example.

Setup. Let $n = 2$. Our trust region subproblem is:

$$\min_{p \in \mathbb{R}^2} m_k(p) = g_k^T p + \frac{1}{2} p^T B_k p \quad \text{s.t.} \quad p_1^2 + p_2^2 \leq \Delta_k^2$$

(We drop $f(x_k)$ since it's a constant.)

Rewrite the constraint as $h(p) = p_1^2 + p_2^2 - \Delta_k^2 \leq 0$.

The Lagrangian:

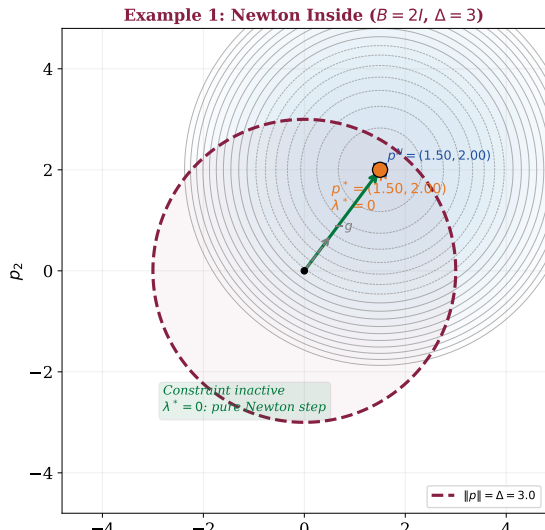
$$\mathcal{L}(p, \lambda) = g_k^T p + \frac{1}{2} p^T B_k p + \frac{\lambda}{2} (p_1^2 + p_2^2 - \Delta_k^2) = g_k^T p + \frac{1}{2} p^T (B_k + \lambda I) p - \frac{\lambda}{2} \Delta_k^2$$

KKT conditions (necessary for a local min when LICQ holds):

- ① $\nabla_p \mathcal{L} = g_k + (B_k + \lambda I)p = 0$ (stationarity)
- ② $\lambda(p_1^2 + p_2^2 - \Delta_k^2) = 0, \quad \lambda \geq 0$ (complementarity, dual feasibility)
- ③ $p_1^2 + p_2^2 \leq \Delta_k^2$ (primal feasibility)

We use the **same data** throughout Examples 1–2: $g_k = (-3, -4)^T$ and $B_k = 2I = \begin{pmatrix} 2 & 0 \\ 0 & 2 \end{pmatrix}$.

Example 1: Newton Step Inside ($B_k \succ 0$, $\|p^N\| < \Delta_k$)



Data: $g_k = (-3, -4)^T$, $B_k = 2I$, $\Delta_k = 3$.

Try $\lambda = 0$: $2Ip = (3, 4)^T \Rightarrow p^* = (3/2, 2)^T$.

$\|p^*\| = \sqrt{9/4 + 4} = 5/2 = 2.5 \leq 3$. ✓ **Inside!**

Complementarity: $0 \cdot (\cdot) = 0$. ✓

PSD: $B_k + 0 \cdot I = 2I \succ 0$. ✓

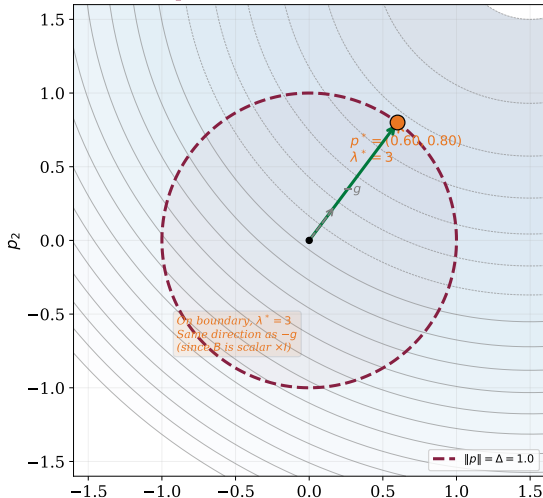
Conclusion: $p^* = (3/2, 2)^T$, $\lambda^* = 0$.

Constraint inactive — the Newton step is the solution.

Circular contours since $B_k = 2I$.

Example 2: Newton Step Outside ($B_k \succ 0$, $\|p^N\| > \Delta_k$)

Example 2: Newton Outside ($B = 2I$, $\Delta = 1$)



Same g_k , B_k , but $\Delta_k = 1$. Now $\|p^N\| = 5/2 > 1$.
Outside!

Stationarity with $\lambda > 0$:

$$p(\lambda) = \frac{1}{2 + \lambda}(3, 4)^T$$

Complementarity $\|p(\lambda)\| = 1$:

$$\frac{5}{2 + \lambda} = 1 \Rightarrow \boxed{\lambda^* = 3}$$

$$\boxed{p^* = (3/5, 4/5)^T}, \quad \|p^*\| = 1 \checkmark$$

Key: λ^* shifts B to $5I$, scaling p^N down to the boundary. Same direction as $-g$ since $B = 2I$.

Example 2: Verifying KKT

Verify all KKT conditions with $\lambda^* = 3$, $p^* = (3/5, 4/5)^T$:

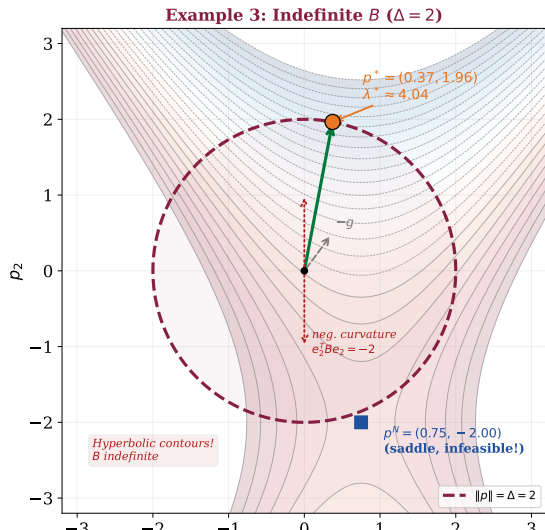
- ➊ **Stationarity:** $(B_k + 3I)p^* = 5I \cdot \frac{1}{5}(3, 4)^T = (3, 4)^T = -g_k$. ✓
- ➋ **Complementarity:** $\lambda^* = 3 > 0$ and $\|p^*\| = 1 = \Delta_k$, so $\lambda^*(\Delta_k - \|p^*\|) = 0$. ✓
- ➌ **Dual feasibility:** $\lambda^* = 3 \geq 0$. ✓
- ➍ **Primal feasibility:** $\|p^*\| = 1 \leq 1 = \Delta_k$. ✓

Is this a minimum? Check the second-order condition:

$$B_k + \lambda^*I = 2I + 3I = 5I \succ 0 \quad \checkmark$$

Observation: Since $B_k = 2I$ is a scalar multiple of I , all components of $p(\lambda)$ shrink at the **same rate**. So p^* points in the *same direction* as p^N — it's just a shorter step. This won't be true for general B_k (the components shrink at different rates).

Example 3: Indefinite B_k — KKT Isn't Enough!



$B_k = \text{diag}(4, -2)$, $g_k = (-3, -4)^T$, $\Delta_k = 2$.

Try $\lambda = 0$: $p = (3/4, -2)^T$.

$\|p\| \approx 2.14 > 2$. **Outside!**

And B_k has eigenvalue -2 : this point is a **saddle**, not a minimizer!

Note the **hyperbolic contours**: m_k curves *down* along e_2 .

What went wrong? KKT doesn't distinguish saddles from minima when B_k is indefinite. We need:

$B_k + \lambda I \succeq 0$. Since $\lambda_{\min}(B_k) = -2$, we need $\lambda \geq 2$.

Example 3: Finding the Correct Solution

We need $\lambda^* \geq 2$ (to make $B_k + \lambda I \succeq 0$) and $\|p(\lambda)\| = 2$ (boundary).

Stationarity: $p(\lambda) = \begin{pmatrix} 3/(4+\lambda) \\ 4/(\lambda-2) \end{pmatrix}$ (requires $\lambda > 2$)

Secular equation: $\frac{9}{(4+\lambda)^2} + \frac{16}{(\lambda-2)^2} = 4$

- $\lambda = 3$: $\frac{9}{49} + \frac{16}{1} = 0.18 + 16 = 16.18$ (way too big — p_2 component is huge near $\lambda = 2$)
- $\lambda = 4$: $\frac{9}{64} + \frac{16}{4} = 0.14 + 4 = 4.14$
- $\lambda = 5$: $\frac{9}{81} + \frac{16}{9} = 0.11 + 1.78 = 1.89$
- Bisection in $[4, 5]$: $\lambda^* \approx 4.04$, $p^* \approx (0.373, 1.96)^T$, $\|p^*\| \approx 2.0$.

Check PSD: $B_k + \lambda^* I \approx \text{diag}(8.04, 2.04) \succ 0$. ✓

Key lesson: λ^* must be large enough to “cancel” the negative curvature in B_k , making $B_k + \lambda^* I \succeq 0$. This is the condition standard KKT misses.

What We Learned from the Examples

Applying KKT to the TRS produced three conditions:

$$\textcircled{1} (B_k + \lambda I)p = -g_k \quad (\text{stationarity of Lagrangian})$$

$$\textcircled{2} \lambda(\|p\|^2 - \Delta_k^2) = 0, \lambda \geq 0 \quad (\text{complementarity + dual feasibility})$$

But Example 3 showed that KKT alone is **not sufficient** — it can return saddle points. We need a third condition:

$$\textcircled{3} B_k + \lambda I \succeq 0 \quad (\text{positive semidefiniteness})$$

This ensures the Lagrangian is **convex** in p , so the KKT point is a **global** minimizer.

Exercise

Repeat Examples 1–2 with $B_k = \text{diag}(2, 8)$ (not a scalar multiple of I). Using $g_k = (-3, -4)^T$ and $\Delta_k = 1$:

$$\textcircled{1} \text{ Show that } p^N = (3/2, 1/2)^T \text{ with } \|p^N\| = \sqrt{10}/2 > 1.$$

$$\textcircled{2} \text{ Write } p(\lambda) = (3/(2 + \lambda), 4/(8 + \lambda))^T \text{ and solve } \|p(\lambda)\| = 1.$$

$$\textcircled{3} \text{ Verify that } p^* \text{ is } \mathbf{not} \text{ in the same direction as } p^N.$$

Where Does the PSD Condition Come From?

From Examples 1–3, we saw that KKT gives conditions 1–2. But **why** do we need $B_k + \lambda I \succeq 0$?

The Lagrangian tells us. At a KKT point, stationarity says:

$$\nabla_p \mathcal{L} = g_k + (B_k + \lambda I)p = 0 \implies p \text{ is a } \mathbf{stationary point} \text{ of } \mathcal{L}(p, \lambda)$$

But a stationary point could be a **min, max, or saddle** of $\mathcal{L}$ in p !

- If $B_k + \lambda I \succ 0$: $\mathcal{L}$ is **convex** in $p \Rightarrow$ stationary point is a **global minimum** of $\mathcal{L}$.
- If $B_k + \lambda I$ has a negative eigenvalue: $\mathcal{L}$ is **nonconvex** $\Rightarrow$ stationary point could be a **saddle or max**.

Since $\mathcal{L}(p, \lambda) = m_k(p) + \frac{\lambda}{2}(\|p\|^2 - \Delta_k^2)$, minimizing $\mathcal{L}$ over p gives:

$$m_k(p) = \mathcal{L}(p, \lambda) - \frac{\lambda}{2}(\|p\|^2 - \Delta_k^2) \geq \mathcal{L}(p^*, \lambda) - \frac{\lambda}{2}(\|p\|^2 - \Delta_k^2)$$

Complementarity ($\lambda\|p^*\|^2 = \lambda\Delta_k^2$) and feasibility ($\|p\| \leq \Delta_k$) then give $m_k(p) \geq m_k(p^*)$.

Bottom line: The PSD condition ensures the Lagrangian is convex in p , which is what makes KKT sufficient for *global* optimality. Without it, KKT only gives local/saddle points.

Three Cases: Summary Before the Theorem

	Case 1	Case 2	Case 3
	$B_k \succ 0,$ $\ p^N\ \leq \Delta_k$	$B_k \succ 0,$ $\ p^N\ > \Delta_k$	B_k indefinite
Solution	$p^* = p^N$	On boundary	On boundary
λ^*	$= 0$	> 0	$\geq -\lambda_{\min}(B_k)$
Constraint	Inactive	Active	Active
KKT alone?	Sufficient	Sufficient	Not sufficient

In all three cases, conditions 1–3 hold. The theorem we state next proves that these three conditions are *necessary and sufficient* for **global** optimality of the TRS.

Preview: Condition 3 is automatic when $B_k \succ 0$ (Cases 1–2), but becomes the crucial extra ingredient when B_k is indefinite (Case 3).

Solving the TRS: Optimality Conditions

Theorem (Nocedal & Wright, Thm. 4.1)

A vector p^* is a **global** solution of the TRS if and only if p^* is feasible and there exists $\lambda^* \geq 0$ such that:

- ① $(B_k + \lambda^* I) p^* = -g_k$ (stationarity)
- ② $\lambda^*(\Delta_k - \|p^*\|) = 0$ (complementarity)
- ③ $B_k + \lambda^* I \succeq 0$ (positive semidefiniteness)

Recall the TRS we are solving:

$$\min_{p \in \mathbb{R}^n} \underbrace{f(x_k) + g_k^T p + \frac{1}{2} p^T B_k p}_{m_k(p)} \quad \text{subject to} \quad \|p\|_2 \leq \Delta_k$$

This theorem gives **necessary and sufficient** conditions for a *global* optimum — remarkable because m_k may be **nonconvex** (when B_k has negative eigenvalues).

Theorem 4.1: What Each Variable Means

- $p^* \in \mathbb{R}^n$: the **optimal step** — the direction and distance we should move from x_k .
- $g_k = \nabla f(x_k) \in \mathbb{R}^n$: the **gradient** of f at the current iterate.
- $B_k \in \mathbb{R}^{n \times n}$: a **symmetric** matrix approximating the Hessian $\nabla^2 f(x_k)$. Could be the exact Hessian, or a quasi-Newton approximation (BFGS, SR1). May be **indefinite**.
- $\Delta_k > 0$: the **trust region radius** — how far we allow p to go.
- $\lambda^* \geq 0$: the **Lagrange multiplier** for the constraint $\|p\| \leq \Delta_k$.
- $I \in \mathbb{R}^{n \times n}$: the **identity matrix**.
- $B_k + \lambda^* I \succeq 0$: means all eigenvalues of $B_k + \lambda^* I$ are ≥ 0 , i.e., this matrix is **positive semidefinite**. If B_k has eigenvalues $\mu_1 \leq \dots \leq \mu_n$, this requires $\lambda^* \geq -\mu_1 = \max(0, -\lambda_{\min}(B_k))$.

Theorem 4.1: What Each Condition Means

- ❶ **Stationarity:** $(B_k + \lambda^* I)p^* = -g_k$.

This is the gradient of the **Lagrangian** $\mathcal{L}(p, \lambda) = m_k(p) + \frac{\lambda}{2}(\|p\|^2 - \Delta_k^2)$ set to zero:

$$\nabla_p \mathcal{L} = g_k + B_k p + \lambda p = g_k + (B_k + \lambda I)p = 0$$

So p^* is a *stationary point* of the Lagrangian.

- ❷ **Complementarity:** $\lambda^*(\Delta_k - \|p^*\|) = 0$.

Either $\lambda^* = 0$ (constraint is inactive, p^* is the unconstrained minimizer inside the ball) or $\|p^*\| = \Delta_k$ (constraint is active, p^* is on the boundary). *Not both idle.*

- ❸ **Positive semidefiniteness:** $B_k + \lambda^* I \succeq 0$.

This ensures the Lagrangian is **convex** in p , so stationarity gives a *global* (not just local) minimizer of $\mathcal{L}$ over all p . This is the condition that goes beyond standard KKT.

Theorem 4.1: A Concrete Example

Setup. Let $n = 2$, $f(x_k) = 5$, and:

$$g_k = \begin{pmatrix} 2 \\ 1 \end{pmatrix}, \quad B_k = \begin{pmatrix} 3 & 0 \\ 0 & -1 \end{pmatrix}, \quad \Delta_k = 1$$

Note: B_k is **indefinite** ($\lambda_{\min}(B_k) = -1$). The unconstrained minimizer of m_k is $p^N = -B_k^{-1}g_k = (-2/3, 1)^T$, with $\|p^N\| \approx 1.20 > \Delta_k$. So the Newton step is *outside* the trust region.

Applying the theorem. We need $\lambda^* \geq -\lambda_{\min}(B_k) = 1$ (condition 3). Since $\|p^N\| > \Delta_k$, the constraint must be active: $\|p^*\| = \Delta_k = 1$ and $\lambda^* > 0$ (condition 2).

Condition 1 gives: $p^*(\lambda) = -(B_k + \lambda I)^{-1}g_k = \left(\frac{-2}{3+\lambda}, \frac{-1}{-1+\lambda} \right)^T$.

We find λ^* by solving $\|p^*(\lambda)\| = 1$:

$$\frac{4}{(3+\lambda)^2} + \frac{1}{(\lambda-1)^2} = 1$$

Numerically: $\lambda^* \approx 2.09$, giving $p^* \approx (-0.393, -0.919)^T$ with $\|p^*\| = 1$. ✓

Example Continued: Checking All Conditions

With $\lambda^* \approx 2.09$ and $p^* \approx (-0.393, -0.919)^T$:

Condition 1 (stationarity):

$$(B_k + \lambda^* I)p^* = \begin{pmatrix} 5.09 & 0 \\ 0 & 1.09 \end{pmatrix} \begin{pmatrix} -0.393 \\ -0.919 \end{pmatrix} \approx \begin{pmatrix} -2 \\ -1 \end{pmatrix} = -g_k \quad \checkmark$$

Condition 2 (complementarity): $\lambda^* = 2.09 > 0$ and $\|p^*\| = 1 = \Delta_k$.

So $\lambda^*(\Delta_k - \|p^*\|) = 2.09 \cdot 0 = 0$. $\checkmark$

Condition 3 (PSD): eigenvalues of $B_k + \lambda^* I$ are $3 + 2.09 = 5.09$ and $-1 + 2.09 = 1.09$. Both ≥ 0 . $\checkmark$

Key observation: λ^* had to be large enough to (a) make $B_k + \lambda^* I$ positive semidefinite ($\lambda^* \geq 1$), and (b) shrink the step $p^*(\lambda)$ to fit inside the trust region ($\|p^*\| = \Delta_k$). The multiplier λ^* plays *both* roles simultaneously.

Proof of Theorem 4.1: Setup

We prove both directions of the “if and only if.”

Problem: $\min_p m_k(p) = f_k + g_k^T p + \frac{1}{2} p^T B_k p$ s.t. $\|p\|^2 \leq \Delta_k^2$.

(We write the constraint as $\|p\|^2 \leq \Delta_k^2$ for differentiability.)

Lagrangian:

$$\mathcal{L}(p, \lambda) = f_k + g_k^T p + \frac{1}{2} p^T B_k p + \frac{\lambda}{2} (\|p\|^2 - \Delta_k^2) = f_k + g_k^T p + \frac{1}{2} p^T (B_k + \lambda I) p - \frac{\lambda}{2} \Delta_k^2$$

Lagrangian dual function: $q(\lambda) = \inf_{p \in \mathbb{R}^n} \mathcal{L}(p, \lambda)$.

If $B_k + \lambda I \succeq 0$, the infimum is attained at $p(\lambda) = -(B_k + \lambda I)^\dagger g_k$, provided $g_k \in \text{Range}(B_k + \lambda I)$. Otherwise $q(\lambda) = -\infty$.

Recall: the pseudoinverse $A^\dagger$. If $A = Q \text{diag}(\sigma_1, \dots, \sigma_r, 0, \dots, 0) Q^T$ (eigendecomposition, $\sigma_i > 0$), then $A^\dagger = Q \text{diag}(1/\sigma_1, \dots, 1/\sigma_r, 0, \dots, 0) Q^T$. It inverts A on its range and maps the null space to zero. When A is invertible, $A^\dagger = A^{-1}$.

Proof: Sufficiency (Conditions 1–3 $\Rightarrow$ Global Optimality)

Claim: If p^* is feasible and $\lambda^* \geq 0$ satisfy conditions 1–3, then p^* is a global solution.

Proof. Let p be any feasible point ($\|p\| \leq \Delta_k$). We show $m_k(p) \geq m_k(p^*)$.

Step 1 (Rewrite m_k using λ^*): Add and subtract $\frac{\lambda^*}{2} \|p\|^2$:

$$m_k(p) = f_k + g_k^T p + \frac{1}{2} p^T \underbrace{(B_k + \lambda^* I)}_{\succeq 0 \text{ by cond. 3}} p - \frac{\lambda^*}{2} \|p\|^2$$

Why? This isolates the PSD matrix $B_k + \lambda^* I$, letting us use convexity.

Step 2 (Identify convex part): Define $\varphi(p) := g_k^T p + \frac{1}{2} p^T (B_k + \lambda^* I) p$.

By **condition 3**, $(B_k + \lambda^* I) \succeq 0$, so φ is **convex** in p .

Step 3 (Use condition 1): $\nabla_p \varphi(p^*) = g_k + (B_k + \lambda^* I) p^* = 0$ by cond. 1.

A stationary point of a convex function is a **global minimizer**:

$$\varphi(p) \geq \varphi(p^*) \quad \text{for all } p \in \mathbb{R}^n$$

Proof: Sufficiency (Continued)

Step 4 (Combine): Substituting $\varphi(p) \geq \varphi(p^*)$ back:

$$\begin{aligned} m_k(p) &= f_k + \varphi(p) - \frac{\lambda^*}{2} \|p\|^2 \\ &\geq f_k + \varphi(p^*) - \frac{\lambda^*}{2} \|p\|^2 \quad (\text{Step 3: convexity}) \\ &= m_k(p^*) + \frac{\lambda^*}{2} (\|p^*\|^2 - \|p\|^2) \end{aligned}$$

Step 5 (Use condition 2 to finish): Need $\|p^*\|^2 \geq \|p\|^2$ (after multiplying by $\lambda^* \geq 0$).

By **complementarity**: $\lambda^*(\Delta_k - \|p^*\|) = 0$, so $\lambda^*\|p^*\|^2 = \lambda^*\Delta_k^2$.

By **feasibility** of p : $\|p\| \leq \Delta_k$, so $\lambda^*\|p\|^2 \leq \lambda^*\Delta_k^2$.

Together: $\lambda^*\|p^*\|^2 = \lambda^*\Delta_k^2 \geq \lambda^*\|p\|^2$.

$$m_k(p) \geq m_k(p^*) + \underbrace{\frac{\lambda^*}{2} (\|p^*\|^2 - \|p\|^2)}_{\geq 0} \geq m_k(p^*)$$

Since p was *any* feasible point, p^* is a global minimizer. □

Proof: Necessity (Global Optimality $\Rightarrow$ Conditions 1–3)

Claim: If p^* is a global solution of the TRS, then there exists $\lambda^* \geq 0$ satisfying conditions 1–3.

Case A: p^* is in the interior ($\|p^*\| < \Delta_k$).

Then p^* is an unconstrained local (hence global) minimizer of m_k :

$$\nabla m_k(p^*) = g_k + B_k p^* = 0 \quad \Rightarrow \quad B_k p^* = -g_k$$

Set $\lambda^* = 0$. Then:

- ① $(B_k + 0 \cdot I)p^* = B_k p^* = -g_k$. ✓
- ② $\lambda^*(\Delta_k - \|p^*\|) = 0 \cdot (\Delta_k - \|p^*\|) = 0$. ✓
- ③ $B_k + 0 \cdot I = B_k \succeq 0$. This must hold because p^* is an unconstrained minimizer of the quadratic m_k , which requires B_k to be PSD (second-order necessary conditions). ✓

Note: In this case, the trust region constraint is inactive — we're taking the full Newton step.

Proof: Necessity (Case B — Boundary, $\|p^*\| = \Delta_k$)

Step 1 (KKT gives conditions 1–2 and $\lambda^* \geq 0$):

The constraint $h(p) = \|p\|^2 - \Delta_k^2 \leq 0$ satisfies LICQ at p^* (since $\nabla h(p^*) = 2p^* \neq 0$).

By the KKT conditions for a minimizer with inequality constraints, there exists $\lambda^* \geq 0$ such that:

$$\nabla_p \mathcal{L}|_{p^*} = g_k + B_k p^* + \lambda^* p^* = 0 \implies (B_k + \lambda^* I) p^* = -g_k \quad \checkmark \text{ (cond. 1)}$$

Since $\|p^*\| = \Delta_k$: $\lambda^* (\|p^*\|^2 - \Delta_k^2) = \lambda^* \cdot 0 = 0 \quad \checkmark \text{ (cond. 2)}$

What we have so far: Conditions 1, 2, and $\lambda^* \geq 0$ all follow from standard KKT (dual feasibility is *part of* the KKT conditions for inequality constraints).

What remains: Only one thing — and it goes **beyond** standard KKT:

- $B_k + \lambda^* I \succeq 0$ (the PSD condition)

Proof: Necessity (The PSD Condition — Algebra)

Step 2 ($B_k + \lambda^* I \succeq 0$): This is the only part that goes beyond KKT.

Since p^* is a *global* minimizer on $\|p\| \leq \Delta_k$, it also minimizes m_k on the sphere $\|p\| = \Delta_k$. So $m_k(p) \geq m_k(p^*)$ for all p with $\|p\| = \Delta_k$.

Let $d = p - p^*$. Expanding $m_k(p) - m_k(p^*)$:

$$m_k(p) - m_k(p^*) = \underbrace{(g_k + B_k p^*)^T}_{\text{use stationarity}} d + \frac{1}{2} d^T B_k d$$

Use λ^* from KKT: Stationarity gives $g_k + B_k p^* = -\lambda^* p^*$, so:

$$m_k(p) - m_k(p^*) = -\lambda^* (p^*)^T d + \frac{1}{2} d^T B_k d$$

Use that both p, p^* are on the sphere: $\|p^* + d\|^2 = \|p^*\|^2 = \Delta_k^2$ gives $(p^*)^T d = -\frac{1}{2} \|d\|^2$, so:

$$m_k(p) - m_k(p^*) = \frac{\lambda^*}{2} \|d\|^2 + \frac{1}{2} d^T B_k d = \boxed{\frac{1}{2} d^T (B_k + \lambda^* I) d \geq 0}$$

Both ingredients — the specific λ^* from KKT *and* the sphere geometry — combine to produce $(B_k + \lambda^* I)$.

Proof: Necessity (The PSD Condition — Conclusion)

We have shown: $(p - p^*)^T (B_k + \lambda^* I)(p - p^*) \geq 0$ for all p with $\|p\| = \Delta_k$.

From specific vectors to PSD: We need $v^T (B_k + \lambda^* I)v \geq 0$ for *all* $v \in \mathbb{R}^n$.

Since $v \mapsto v^T (B_k + \lambda^* I)v$ is **homogeneous of degree 2**, it suffices to check all *unit* vectors w .

The set of normalized directions $\left\{ w = \pm \frac{p - p^*}{\|p - p^*\|} : \|p\| = \Delta_k, p \neq p^* \right\}$ is **dense** on the unit sphere. (It covers all directions except those perpendicular to p^* , which are limits.)

We know $w^T (B_k + \lambda^* I)w \geq 0$ on this dense set. Since $w \mapsto w^T (B_k + \lambda^* I)w$ is **continuous**:

$$w^T (B_k + \lambda^* I)w \geq 0 \quad \text{for all } w \in \mathbb{R}^n$$

This is the definition of $B_k + \lambda^* I \succeq 0$ (condition 3). □

Proof Summary

Direction	Key idea
Sufficiency	Rewrite m_k by adding/subtracting $\frac{\lambda^*}{2} \ p\ ^2$. Cond. 3 makes the quadratic part convex; cond. 1 gives its global min; cond. 2 closes the gap.
Necessity	Interior: $\lambda^* = 0$, standard unconstrained SONC. Boundary: KKT $\Rightarrow$ conds 1–2 and $\lambda^* \geq 0$ (dual feasibility is part of KKT). PSD (cond. 3) follows from $m_k(p) \geq m_k(p^*)$ on the sphere + density of directions (N&W p. 91).

Practical takeaway: Finding p^* reduces to a **1D search**: find $\lambda^* \geq 0$ with $\|p(\lambda)\| = \Delta_k$, where $p(\lambda) = -(B_k + \lambda I)^{-1} g_k$.

The search itself is cheap (2–3 Newton iterations), but each evaluation of $\|p(\lambda)\|$ requires solving an $n \times n$ linear system — cost $O(n^3)$ per factorization. Total: $O(n^3)$, not $O(n)$.

How do we solve this, and when does it break down? Next slides.

Why Is the TRS Tractable Despite Nonconvexity?

The quadratic model m_k can be nonconvex (when B_k is indefinite), yet conditions 1–3 give a *complete* characterization of the global solution. Why?

The answer: The TRS is a QCQP with a **single** quadratic constraint ($\|p\|^2 \leq \Delta_k^2$). The **S-Lemma** (Yakubovich 1971) guarantees that such problems have **zero duality gap**:

S-Lemma (informal)

Given symmetric matrices A, B with $\exists \bar{x} : \bar{x}^T B \bar{x} > 0$, the implication

$$x^T B x \geq 0 \implies x^T A x \geq 0$$

holds for all x **if and only if** $\exists \lambda \geq 0$ such that $A - \lambda B \succeq 0$.

Consequence for the TRS: Strong duality holds $\implies$ the KKT conditions (with PSD) are necessary *and* sufficient, giving us Theorem 4.1.

Contrast: With **two or more** quadratic constraints, the S-Lemma does not apply, strong duality can fail, and the problem becomes NP-hard in general. The TRS is tractable precisely because it has one quadratic constraint.

Refs: Yakubovich (1971); Pólik & Terlaky (2007), *A Survey of the S-Lemma*, SIAM Review.

Linear Algebra Trick: Diagonalizing $B_k + \lambda I$

Key idea: If we eigendecompose B_k , then shifting by λI just shifts the eigenvalues.

Let $B_k = Q\Lambda Q^T$ where $\Lambda = \text{diag}(\lambda_1, \dots, \lambda_n)$ and $Q = [q_1 \mid \dots \mid q_n]$ is orthogonal. Then:

$$B_k + \lambda I = Q\Lambda Q^T + \lambda Q Q^T = Q(\Lambda + \lambda I)Q^T = Q \text{diag}(\lambda_1 + \lambda, \dots, \lambda_n + \lambda) Q^T$$

Inverting (when all $\lambda_j + \lambda \neq 0$):

$$(B_k + \lambda I)^{-1} = Q \text{diag}\left(\frac{1}{\lambda_1 + \lambda}, \dots, \frac{1}{\lambda_n + \lambda}\right) Q^T$$

Applying to g_k : Write g_k in the eigenbasis as $g_k = \sum_j (q_j^T g_k) q_j$. Then:

$$p(\lambda) = -(B_k + \lambda I)^{-1} g_k = - \sum_{j=1}^n \frac{q_j^T g_k}{\lambda_j + \lambda} q_j$$

Each component of p (in the eigenbasis) is scaled by $1/(\lambda_j + \lambda)$. As λ increases, every component shrinks — so $\|p(\lambda)\|$ decreases monotonically. This is the foundation for the secular equation.

Solving the TRS: The Secular Equation (“Easy Case”)

From the previous slide, $p(\lambda) = -\sum_j \frac{q_j^T g_k}{\lambda_j + \lambda} q_j$. Since the q_j are orthonormal:

$$\|p(\lambda)\|^2 = \sum_{j=1}^n \frac{(q_j^T g_k)^2}{(\lambda_j + \lambda)^2}$$

The secular equation: Find λ^* such that $\|p(\lambda^*)\| = \Delta_k$:

$$\phi(\lambda) := \sum_{j=1}^n \frac{(q_j^T g_k)^2}{(\lambda_j + \lambda)^2} = \Delta_k^2$$

Why this works (when $q_1^T g_k \neq 0$):

- $\phi(\lambda)$ is **smooth** and **monotonically decreasing** on $\lambda \in (-\lambda_1, \infty)$
- As $\lambda \rightarrow (-\lambda_1)^+$: the $j = 1$ term $\frac{(q_1^T g_k)^2}{(\lambda_1 + \lambda)^2} \rightarrow \infty$, so $\phi(\lambda) \rightarrow \infty$
- As $\lambda \rightarrow \infty$: every term $\rightarrow 0$, so $\phi(\lambda) \rightarrow 0$
- By the IVT: $\exists!$ root λ^* with $\phi(\lambda^*) = \Delta_k^2$

Algorithm 4.3: Newton on the Secular Equation

Apply Newton's method to $\psi(\lambda) = 1/\|p(\lambda)\| - 1/\Delta_k = 0$.

What we need: $\psi'(\lambda)$. Differentiate $\|p(\lambda)\|^2$ w.r.t. λ :

$$\frac{d}{d\lambda} \|p\|^2 = 2p^T \frac{dp}{d\lambda}, \quad \text{where} \quad \frac{dp}{d\lambda} = -(B_k + \lambda I)^{-1} p(\lambda)$$

Key trick: If we already have Cholesky $B_k + \lambda I = R^T R$ and p from $R^T R p = -g_k$, then solve one more triangular system $R^T q = p$ to get:

$$\frac{d}{d\lambda} \|p\|^2 = -2 \underbrace{p^T (B_k + \lambda I)^{-1} p}_{=\|R^{-T} p\|^2 = \|q\|^2} = -2\|q\|^2$$

No second derivatives needed — just one extra $O(n^2)$ triangular solve.

Newton update:

$$\lambda_{l+1} = \lambda_l + \frac{\|p_l\|^2 (\|p_l\| - \Delta_k)}{\|q_l\|^2 \Delta_k}$$

Converges in **2–3 iterations** typically. Each iteration: 1 Cholesky ($O(n^3)$) + 2 triangular solves ($O(n^2)$).

The Hard Case (N&W, pp. 87–88)

The secular equation approach works because $\|p(\lambda)\| \rightarrow \infty$ as $\lambda \rightarrow (-\lambda_1)^+$, guaranteeing a root. But this relies on $q_1^T g_k \neq 0$.

The Hard Case: $q_1^T g_k = 0$

When g_k is **orthogonal** to the eigenspace of $\lambda_{\min}(B_k)$, the $j = 1$ term vanishes and $\|p(\lambda)\|$ stays bounded as $\lambda \rightarrow -\lambda_1$:

$$\lim_{\lambda \rightarrow -\lambda_1^+} \|p(\lambda)\|^2 = \sum_{j: \lambda_j \neq \lambda_1} \frac{(q_j^T g_k)^2}{(\lambda_j - \lambda_1)^2} \quad (\text{finite!})$$

If this limit is $< \Delta_k^2$, there is **no** $\lambda \in (-\lambda_1, \infty)$ with $\|p(\lambda)\| = \Delta_k$. The secular equation has no root!

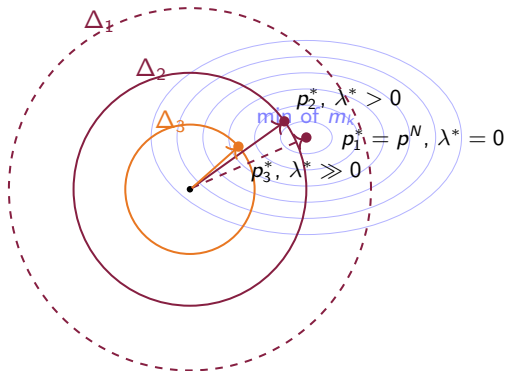
Solution: Set $\lambda^* = -\lambda_1$ and add a null-space component:

$$p^* = - \underbrace{\sum_{j: \lambda_j \neq \lambda_1} \frac{q_j^T g_k}{\lambda_j - \lambda_1} q_j}_{\text{pseudoinverse solution}} + \underbrace{\tau q_1}_{\text{null space}}, \quad \text{choose } \tau \text{ so that } \|p^*\| = \Delta_k$$

Satisfies Theorem 4.1: $(B_k - \lambda_1 I)q_1 = 0$, so the τq_1 term vanishes in stationarity. In practice, nearly hard cases ($q_1^T g_k \approx 0$) are more common and cause ill-conditioning.

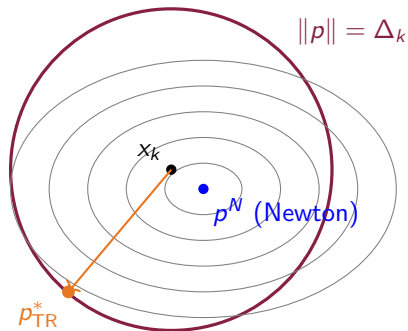
Extreme example: $B_k = -cI$ ($c > 0$), $g_k = 0$. Then $m(p) = -\frac{c}{2}\|p\|^2$ and every boundary point is optimal. Here $q_j^T g_k = 0$ for all j , $\lambda^* = c$, $B_k + \lambda^* I = 0$, and $p^* = \tau q$ for any unit vector q with $\tau = \Delta_k$. Note: all eigenvalues of B_k must be equal; if they differ, the most negative direction is preferred.

Three Radii: How p^* Depends on Δ_k (N&W Fig. 4.2)



As Δ_k shrinks: λ^* increases, p^* gets shorter, and the direction rotates toward $-g_k$. In the limit $\Delta_k \rightarrow 0$:
 $p^* \rightarrow -\frac{\Delta_k}{\|g_k\|} g_k$ (steepest descent, like the Cauchy point).

Geometric View of the TRS



Cases:

$\|p^N\| \leq \Delta_k$: take Newton step

$$(\lambda^* = 0)$$

$\|p^N\| > \Delta_k$: solution on boundary

$$(\lambda^* > 0)$$

B_k indefinite: solution on boundary

$$(\lambda^* \geq -\lambda_{\min}(B_k))$$

The Dual Viewpoint: Regularization

The TRS optimality conditions reveal a duality:

Trust Region (Primal)

$$\min_p m_k(p) \quad \text{s.t.} \quad \|p\| \leq \Delta$$

ℓ_2 Regularization (Dual)

$$\min_p m_k(p) + \frac{\lambda}{2} \|p\|^2$$

For each $\Delta > 0$ there exists $\lambda(\Delta) \geq 0$ yielding the same solution, and vice versa. Strong duality holds here **not** by Slater's condition (which requires a convex objective), but by a special result for problems with a single quadratic constraint (the S-lemma).

Both give: $(B_k + \lambda I)p = -g_k$.

This is the Levenberg–Marquardt interpretation: adding λI to B_k simultaneously:

- Regularizes the system (makes it PD when $\lambda > -\lambda_{\min}(B_k)$)
- Shortens the step (larger $\lambda \Rightarrow$ smaller $\|p\|$)

Check Your Understanding: Theorem 4.1

True or False?

- 1 If $B_k \succ 0$ and $\|p^N\| < \Delta_k$, then $\lambda^* > 0$.

Check Your Understanding: Theorem 4.1

True or False?

- ① If $B_k \succ 0$ and $\|p^N\| < \Delta_k$, then $\lambda^* > 0$.

False. $\lambda^* = 0$: the unconstrained Newton step is the solution.

- ② Standard KKT conditions (stationarity + complementarity + $\lambda \geq 0$) are sufficient for global optimality of the TRS.

Check Your Understanding: Theorem 4.1

True or False?

- ① If $B_k \succ 0$ and $\|p^N\| < \Delta_k$, then $\lambda^* > 0$.

False. $\lambda^* = 0$: the unconstrained Newton step is the solution.

- ② Standard KKT conditions (stationarity + complementarity + $\lambda \geq 0$) are sufficient for global optimality of the TRS.

False. Need the PSD condition too — without it, the KKT point could be a saddle.

- ③ If $B_k \succ 0$, condition 3 ($B_k + \lambda^* I \succeq 0$) is automatically satisfied for any $\lambda^* \geq 0$.

Check Your Understanding: Theorem 4.1

True or False?

- ① If $B_k \succ 0$ and $\|p^N\| < \Delta_k$, then $\lambda^* > 0$.

False. $\lambda^* = 0$: the unconstrained Newton step is the solution.

- ② Standard KKT conditions (stationarity + complementarity + $\lambda \geq 0$) are sufficient for global optimality of the TRS.

False. Need the PSD condition too — without it, the KKT point could be a saddle.

- ③ If $B_k \succ 0$, condition 3 ($B_k + \lambda^* I \succeq 0$) is automatically satisfied for any $\lambda^* \geq 0$.

True. All eigenvalues of B_k are already positive; adding $\lambda^* \geq 0$ only shifts them up.

Check Your Understanding: Theorem 4.1

True or False?

- ① If $B_k \succ 0$ and $\|p^N\| < \Delta_k$, then $\lambda^* > 0$.

False. $\lambda^* = 0$: the unconstrained Newton step is the solution.

- ② Standard KKT conditions (stationarity + complementarity + $\lambda \geq 0$) are sufficient for global optimality of the TRS.

False. Need the PSD condition too — without it, the KKT point could be a saddle.

- ③ If $B_k \succ 0$, condition 3 ($B_k + \lambda^* I \succeq 0$) is automatically satisfied for any $\lambda^* \geq 0$.

True. All eigenvalues of B_k are already positive; adding $\lambda^* \geq 0$ only shifts them up.

Short answer: In one sentence, what goes wrong if $B + \lambda^* I$ has a negative eigenvalue?

Check Your Understanding: Theorem 4.1

True or False?

- ① If $B_k \succ 0$ and $\|p^N\| < \Delta_k$, then $\lambda^* > 0$.

False. $\lambda^* = 0$: the unconstrained Newton step is the solution.

- ② Standard KKT conditions (stationarity + complementarity + $\lambda \geq 0$) are sufficient for global optimality of the TRS.

False. Need the PSD condition too — without it, the KKT point could be a saddle.

- ③ If $B_k \succ 0$, condition 3 ($B_k + \lambda^* I \succeq 0$) is automatically satisfied for any $\lambda^* \geq 0$.

True. All eigenvalues of B_k are already positive; adding $\lambda^* \geq 0$ only shifts them up.

Short answer: In one sentence, what goes wrong if $B + \lambda^* I$ has a negative eigenvalue?

The Lagrangian $\mathcal{L}(p, \lambda^*)$ is nonconvex in p , so the stationary point could be a saddle or maximum — not a global minimum.

How Do We Find the Step p_k in Practice?

The trust region algorithm (next section) needs a step p_k that approximately solves the TRS. We've seen two **exact** approaches:

- ① **Secular equation:** Eigendecompose B_k , solve $\|p(\lambda)\| = \Delta_k$ via Newton. Gives the exact p^* from Theorem 4.1 — but costs $O(n^3)$ and requires detecting the hard case ($q_1^T g_k = 0$).
- ② **Algorithm 4.3 (N&W):** Repeated Cholesky factorizations of $B_k + \lambda I$. Cheaper, but still $O(n^3)$ per factorization.

Two reasons to look for approximate methods:

- **Cost:** For large n (e.g., $n = 10^6$ in bundle adjustment), factorizing B_k is infeasible.
- **Hard case:** The secular equation breaks down when $q_1^T g_k \approx 0$. Approximate methods **sidestep this entirely** — Steihaug-CG handles negative curvature naturally by extending to the boundary.

Good news: For global convergence, we only need **Cauchy decrease**:

$$m_k(0) - m_k(p_k) \geq \frac{1}{2} \|g_k\| \min(\Delta_k, \|g_k\| / \|B_k\|).$$

Cauchy point (cheapest, $O(n)$) $\longrightarrow$ **Dogleg** (1 factorization) $\longrightarrow$ **Steihaug-CG** (matrix-free)

Approach 1: The Cauchy Point

Idea: Don't try to minimize m_k over the full trust region — just minimize along the **steepest descent direction** $-g_k$. This is a 1D optimization, so it's trivially cheap. The result won't be as good as the exact TRS solution, but it's enough to guarantee convergence.

The **Cauchy point** p_k^C minimizes m_k along $-g_k$ within the trust region:

$$p_k^C = -\tau_k \frac{\Delta_k}{\|g_k\|} g_k, \quad \tau_k = \begin{cases} 1 & \text{if } g_k^T B_k g_k \leq 0 \\ \min\left(\frac{\|g_k\|^3}{\Delta_k g_k^T B_k g_k}, 1\right) & \text{otherwise} \end{cases}$$

Derivation: Minimize $m_k(-tg_k) = f_k - t\|g_k\|^2 + \frac{t^2}{2} g_k^T B_k g_k$ over $t > 0$ subject to $t\|g_k\| \leq \Delta_k$.

- If $g_k^T B_k g_k \leq 0$: model is nonincreasing along $-g_k$, so go to boundary.
- If $g_k^T B_k g_k > 0$: unconstrained minimizer is $t^* = \|g_k\|^2 / (g_k^T B_k g_k)$; project into $[0, \Delta_k / \|g_k\|]$.

Cauchy Decrease: The Key Lemma

Theorem (Cauchy Decrease — N&W Lemma 4.3)

The Cauchy point satisfies:

$$m_k(0) - m_k(p_k^C) \geq \frac{1}{2} \|g_k\| \min\left(\Delta_k, \frac{\|g_k\|}{\|B_k\|}\right)$$

Proof. Two cases. Let $\hat{g} = g_k / \|g_k\|$.

Case 1: $g_k^T B_k g_k \leq 0$. Then $\tau_k = 1$ and $p_k^C = -\Delta_k \hat{g}$:

$$m_k(0) - m_k(p_k^C) = \Delta_k \|g_k\| - \frac{\Delta_k^2}{2\|g_k\|^2} g_k^T B_k g_k \geq \Delta_k \|g_k\| \geq \frac{1}{2} \|g_k\| \Delta_k. \checkmark$$

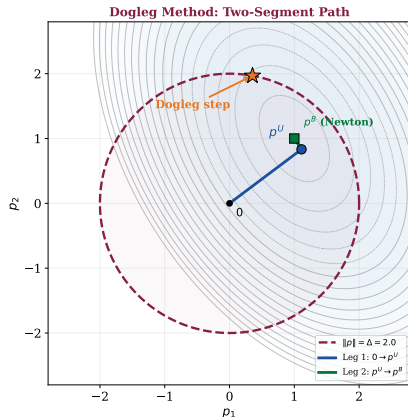
Case 2: $g_k^T B_k g_k > 0$. Sub-case (a): if $t^* \|g_k\| \leq \Delta_k$ (unconstrained min inside TR):

$$m_k(0) - m_k(p_k^C) = \frac{\|g_k\|^4}{2 g_k^T B_k g_k} \geq \frac{\|g_k\|^2}{2 \|B_k\|} = \frac{1}{2} \|g_k\| \cdot \frac{\|g_k\|}{\|B_k\|}. \checkmark$$

Sub-case (b): if $t^* \|g_k\| > \Delta_k$, then $p_k^C = -\Delta_k \hat{g}$ and the bound follows similarly. □

Approach 2: Dogleg Method — The Idea

Motivation: The Cauchy point only searches along $-g_k$ (cheap but poor). The Newton step $p^B = -B^{-1}g$ uses full curvature information (excellent but may be outside the TR). Can we get most of Newton's quality without the full cost of an exact TRS solve?



The dogleg path (requires $B_k \succ 0$):

Leg 1 (steepest descent): $0 \rightarrow p^U$

$$p^U = -\frac{g^T g}{g^T B g} g$$

This is the unconstrained minimizer of m_k along $-g$ (the Cauchy step *without* the TR constraint).

Leg 2 (toward Newton): $p^U \rightarrow p^B$

$$p^B = -B^{-1}g \quad (\text{full Newton step})$$

The name “dogleg” comes from the shape: two straight segments with a bend at p^U , like a dog’s hind leg.

The step: Walk along the dogleg path. If $\|p^B\| \leq \Delta_k$, take the full Newton step. Otherwise, find where the path crosses the TR boundary (a scalar quadratic in τ) and stop there.

Dogleg Method: Why One Factorization Suffices

Key properties of the dogleg path (N&W Lemma 4.2):

- $\|\tilde{p}(\tau)\|$ is **monotonically increasing** along the path (you move further from the origin).
- $m_k(\tilde{p}(\tau))$ is **monotonically decreasing** (the model improves along the path).

$\Rightarrow$ There is a **unique** point where the path crosses the TR boundary, and every point on the path up to the boundary gives at least Cauchy decrease.

Cost comparison — why is this cheaper than exact?

- **Exact (Alg. 4.3):** 2–3 Newton iterations on the secular equation, each requiring a **new** Cholesky factorization of $B_k + \lambda_l I$ (different λ_l each time). Total: 2–3 factorizations.
- **Dogleg:** Factorize B_k **once** to get $p^B = -B^{-1}g$. Compute p^U from g and $g^T Bg$ (just a dot product). Find the boundary crossing by solving a scalar quadratic. Total: **1 factorization**.

Limitation: Requires $B_k \succ 0$ (otherwise p^B doesn't exist as a minimizer, and the Cholesky factorization fails). For indefinite B_k , use Steihaug-CG instead.

Approach 3: Steihaug-Toint Truncated CG

Idea: Run conjugate gradient (CG) to solve the Newton system $B_k p = -g_k$, but **truncate early** if the iterates leave the trust region or encounter negative curvature. CG builds up the solution one direction at a time — at step j , it has searched over a j -dimensional subspace. If the trust region or curvature tells us to stop, we take the best step found so far.

Steihaug-Toint Algorithm

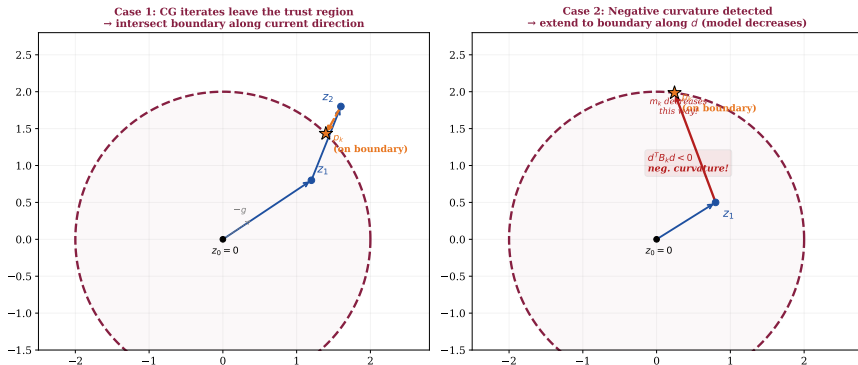
Set $z_0 = 0$, $r_0 = g_k$, $d_0 = -r_0$.

For $j = 0, 1, 2, \dots$:

- ① If $d_j^T B_k d_j \leq 0$ (negative curvature detected):
Find $\tau > 0$ s.t. $\|z_j + \tau d_j\| = \Delta_k$. Return $p = z_j + \tau d_j$.
- ② Set $\alpha_j = r_j^T r_j / (d_j^T B_k d_j)$ and $z_{j+1} = z_j + \alpha_j d_j$.
- ③ If $\|z_{j+1}\| \geq \Delta_k$ (leaving trust region):
Find $\tau > 0$ s.t. $\|z_j + \tau d_j\| = \Delta_k$. Return $p = z_j + \tau d_j$.
- ④ Update: $r_{j+1} = r_j + \alpha_j B_k d_j$.
- ⑤ If $\|r_{j+1}\| < \epsilon_{\text{tol}}$: Return $p = z_{j+1}$ (CG converged).
- ⑥ Set $\beta_{j+1} = r_{j+1}^T r_{j+1} / (r_j^T r_j)$, $d_{j+1} = -r_{j+1} + \beta_{j+1} d_j$.

Steihaug-Toint: How It Works

Steihaug-Toint Truncated CG: Two Termination Cases



Left: CG iterates z_0, z_1, z_2 build up the step inside the trust region. When z_3 would leave the ball, we intersect the current search direction d with the boundary and return p_k on the sphere.

Right: At z_1 , CG finds a direction d with $d^T B_k d < 0$ (negative curvature). Since m_k decreases without bound along d , the best move is to go as far as possible — hit the boundary. No secular equation or hard-case detection needed.

Steihaug-Toint: Properties

Advantages:

- **Matrix-free:** Only requires products $B_k v$ (e.g., via automatic differentiation or finite differences of gradients). Never forms or factorizes B_k .
- **Handles indefinite B_k :** Detects and exploits negative curvature by extending to the boundary — **no hard case** to worry about.
- **Cheap:** $O(n)$ per CG iteration, typically $\ll n$ iterations needed.
- Achieves at least Cauchy decrease (the first CG direction is $-g_k$).

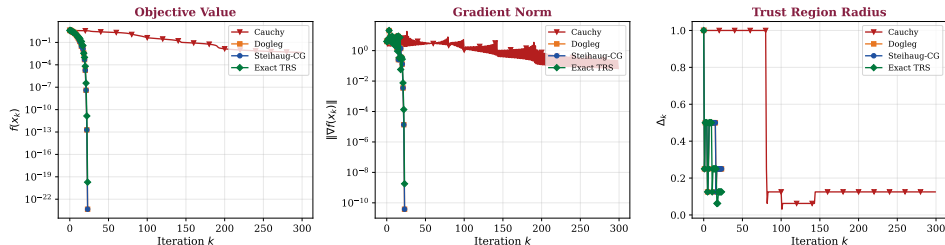
Subproblem solver comparison:

Method	Cost/iter	Requires $B_k \succ 0$?	Quality
Cauchy point	$O(n)$	No	Minimal
Dogleg	$O(n^3)$ factor	Yes	Good
Steihaug CG	$O(jn)$, $j \ll n$	No	Near-optimal
Exact (Alg. 4.3)	$O(n^3)$	No	Optimal

“Exact” = Moré–Sorensen (1983): safeguarded Newton on the secular equation (our Algorithm 4.3) with robust hard-case handling. Uses Cholesky factorization, not eigendecomposition — $O(n^3)$ per iteration but practical. This is what GQTPAR in MINPACK implements.

Numerical Comparison: Rosenbrock Function

Subproblem Solver Comparison on Rosenbrock ($x_0 = (-1, 1)^T$)



All four solvers on $f(x) = 100(x_2 - x_1^2)^2 + (1 - x_1)^2$, $x_0 = (-1, 1)^T$, $\Delta_k^{(0)} = 1$, $\eta = 0.1$.

Cauchy (red ▼): 300 iterations and still at $f \approx 0.005$. Linear convergence is extremely slow on this problem.

Exact, **Steihaug-CG**, **Dogleg**: All converge in **24 iterations** to $f < 10^{-20}$.

Solver Comparison: The Numbers

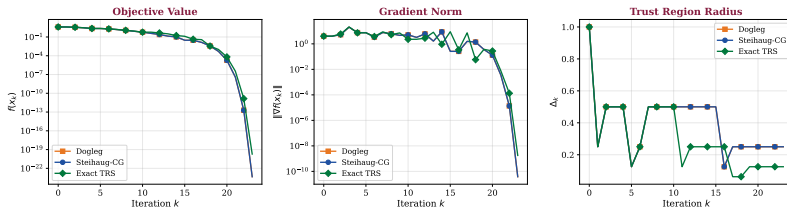
Method	Iterations	Accepted	f evaluations	Final $\ g\ $
Cauchy point	300	297	600	6.6×10^{-2}
Dogleg	24	20	47	3.8×10^{-11}
Steihaug-CG	24	20	47	3.8×10^{-11}
Exact TRS	24	20	47	1.8×10^{-9}

Takeaways:

- Cauchy decrease is sufficient for *convergence* (the theory is correct), but the *rate* matters enormously — $12\times$ more iterations and still not converged.
- Dogleg and Steihaug-CG take **identical steps** on this problem (for $n = 2$ with PD Hessian, the CG path and dogleg path coincide). The exact solver takes **different steps at every iteration** but happens to converge in the same count — see next slide.

Zoomed In: Dogleg vs. Steihaug-CG vs. Exact

Subproblem Solver Comparison (Cauchy excluded) — Rosenbrock, $x_0 = (-1, 1)^T$



Without Cauchy dominating the scale, we can compare the three competitive methods. The convergence curves appear nearly identical, but the underlying steps are **not the same**.

What's really happening:

- **Dogleg = Steihaug-CG** on every step. For $n = 2$, the CG path (first step along $-g$, second toward Newton) is exactly the dogleg path. So these two methods are identical here.
- **Exact $\neq$ Dogleg** on every step. The exact solver finds the true TRS optimum on the boundary; dogleg finds where its two-segment path crosses the boundary — a *different* point. They even disagree on accept/reject (e.g., at $k = 15$, dogleg is rejected while exact is accepted).
- The matching iteration count (24) is **coincidental** for this problem.

For large n : Dogleg still needs $O(n^3)$ (one factorization); Steihaug-CG needs $O(jn)$ with $j \ll n$ (matrix-free); exact needs $2\text{--}3 \times O(n^3)$. For $n \sim 10^6$, only Steihaug-CG is feasible.

Check Your Understanding: Solving the TRS

True or False?

- 1 The Cauchy point always lies on the trust region boundary.

Check Your Understanding: Solving the TRS

True or False?

- 1 The Cauchy point always lies on the trust region boundary.
False. It can be interior when the unconstrained minimizer along $-g_k$ is inside the TR (i.e., $g_k^T B_k g_k > 0$ and the minimizer $\alpha^* < \Delta_k / \|g_k\|$).
- 2 The dogleg method can be used when B_k is indefinite.

Check Your Understanding: Solving the TRS

True or False?

- ① The Cauchy point always lies on the trust region boundary.
False. It can be interior when the unconstrained minimizer along $-g_k$ is inside the TR (i.e., $g_k^T B_k g_k > 0$ and the minimizer $\alpha^* < \Delta_k / \|g_k\|$).
- ② The dogleg method can be used when B_k is indefinite.
False. Requires $B_k \succ 0$. The full step $p^B = -B_k^{-1} g_k$ is only a minimizer when $B_k \succ 0$.

Check Your Understanding: Solving the TRS

True or False?

- ① The Cauchy point always lies on the trust region boundary.
False. It can be interior when the unconstrained minimizer along $-g_k$ is inside the TR (i.e., $g_k^T B_k g_k > 0$ and the minimizer $\alpha^* < \Delta_k / \|g_k\|$).
- ② The dogleg method can be used when B_k is indefinite.
False. Requires $B_k \succ 0$. The full step $p^B = -B_k^{-1} g_k$ is only a minimizer when $B_k \succ 0$.

Short answer: Steihaug-CG encounters a direction d with $d^T B_k d < 0$. What does the algorithm do, and why is this the right response?

Check Your Understanding: Solving the TRS

True or False?

- ❶ The Cauchy point always lies on the trust region boundary.

False. It can be interior when the unconstrained minimizer along $-g_k$ is inside the TR (i.e., $g_k^T B_k g_k > 0$ and the minimizer $\alpha^* < \Delta_k / \|g_k\|$).

- ❷ The dogleg method can be used when B_k is indefinite.

False. Requires $B_k \succ 0$. The full step $p^B = -B_k^{-1} g_k$ is only a minimizer when $B_k \succ 0$.

Short answer: Steihaug-CG encounters a direction d with $d^T B_k d < 0$. What does the algorithm do, and why is this the right response?

It extends to the TR boundary along d . Since $d^T B_k d < 0$, the model m_k *decreases without bound* in that direction. The best we can do within $\|p\| \leq \Delta_k$ is go as far as possible — i.e., hit the boundary.

Predicted vs. Actual Reduction

The key mechanism for adapting Δ_k is comparing:

$$\text{Actual reduction: } \text{ared}_k = f(x_k) - f(x_k + p_k)$$

$$\text{Predicted reduction: } \text{pred}_k = m_k(0) - m_k(p_k) = -g_k^T p_k - \frac{1}{2} p_k^T B_k p_k$$

The **ratio**:

$$\rho_k = \frac{\text{ared}_k}{\text{pred}_k} = \frac{f(x_k) - f(x_k + p_k)}{m_k(0) - m_k(p_k)}$$

- $\rho_k \approx 1$: model is a good approximation $\Rightarrow$ safe to expand Δ_k
- $\rho_k > 0$ but small: model is okay $\Rightarrow$ keep Δ_k
- $\rho_k \leq 0$ or very small: model is poor $\Rightarrow$ shrink Δ_k , reject step

Note: $\text{pred}_k > 0$ whenever p_k reduces the model (guaranteed by Cauchy decrease).

Trust Region Algorithm (Nocedal & Wright, Alg. 4.1)

Algorithm: Basic Trust Region

Given: $\hat{\Delta} > 0$ (max radius), $\Delta_0 \in (0, \hat{\Delta})$, $\eta \in [0, 1/4)$.

For $k = 0, 1, 2, \dots$:

- 1 Obtain p_k by (approximately) solving the TRS:

$$\min_p m_k(p) \quad \text{s.t.} \quad \|p\| \leq \Delta_k$$

- 2 Compute $\rho_k = \frac{f(x_k) - f(x_k + p_k)}{m_k(0) - m_k(p_k)}$

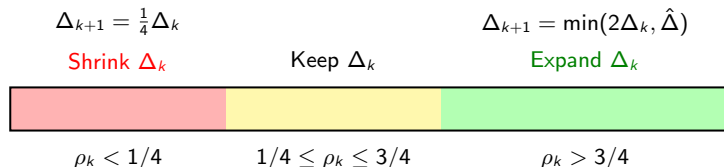
- 3 Update radius:

$$\Delta_{k+1} = \begin{cases} \frac{1}{4}\Delta_k & \text{if } \rho_k < \frac{1}{4} \\ \min(2\Delta_k, \hat{\Delta}) & \text{if } \rho_k > \frac{3}{4} \text{ and } \|p_k\| = \Delta_k \\ \Delta_k & \text{otherwise} \end{cases}$$

- 4 If $\rho_k > \eta$: accept step $x_{k+1} = x_k + p_k$.
Else: reject step $x_{k+1} = x_k$.

How to Choose Δ_k : The Intuition

The radius update in Algorithm 4.1 uses ρ_k as a “thermometer” for model quality:

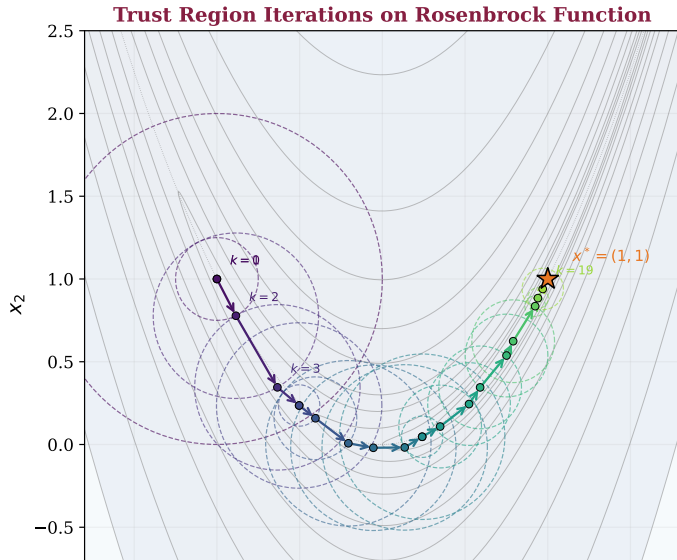


Why these thresholds?

- $\rho_k < 1/4$: Model predicted a decrease but actual decrease was $< 25\%$ of prediction. The model is **unreliable** here $\Rightarrow$ shrink the region where we trust it.
- $\rho_k > 3/4$ **and** $\|p_k\| = \Delta_k$: Model is **accurate** and we hit the boundary $\Rightarrow$ the constraint is limiting us, so expand.
- Expand **only if on the boundary**: If $\|p_k\| < \Delta_k$, the radius isn't the bottleneck, so don't change it.

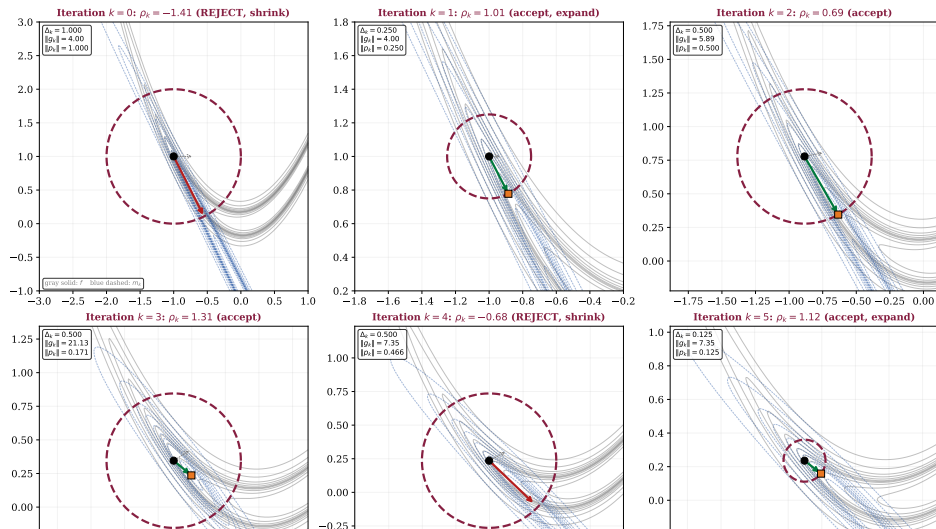
Step acceptance: Accept step if $\rho_k > \eta$ (typically $\eta \in [0, 1/4)$). Even a mediocre model gives *some* decrease.

Trust Region in Action: Rosenbrock Function



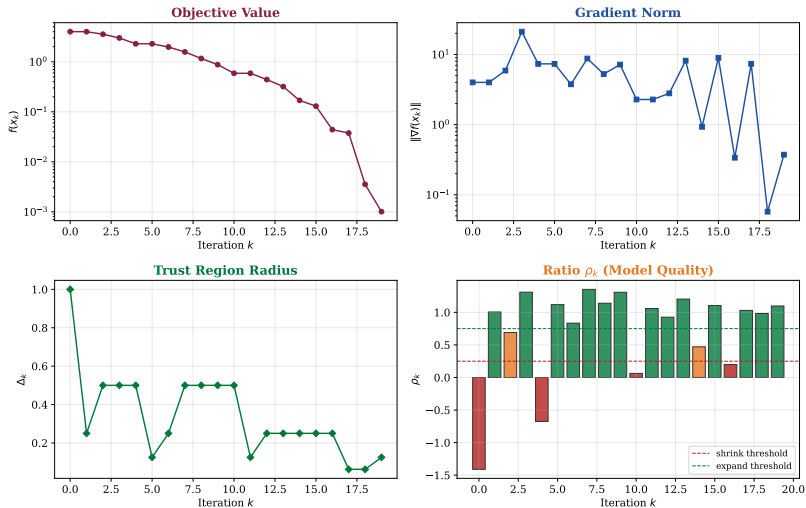
Iteration Details: Model vs. Function

Trust Region Method: Iteration Details on Rosenbrock



Convergence Diagnostics

Trust Region Convergence Diagnostics



Note: rejected steps ($k=0, 4, 10$) cause Δ_k to shrink (bottom-left), and subsequent steps have $\rho_k \approx 1$

Check Your Understanding: The Algorithm

True or False?

- 1 If $\rho_k = 0.9$ but $\|p_k\| < \Delta_k$, we expand Δ_{k+1} .

Check Your Understanding: The Algorithm

True or False?

- ① If $\rho_k = 0.9$ but $\|p_k\| < \Delta_k$, we expand Δ_{k+1} .

False. Expand only if *on the boundary*. If the step is interior, Δ_k isn't constraining progress.

- ② A step with $\rho_k = 0.3$ and $\eta = 0.1$ is accepted, and Δ_k is kept unchanged.

Check Your Understanding: The Algorithm

True or False?

- ① If $\rho_k = 0.9$ but $\|p_k\| < \Delta_k$, we expand Δ_{k+1} .
False. Expand only if *on the boundary*. If the step is interior, Δ_k isn't constraining progress.
- ② A step with $\rho_k = 0.3$ and $\eta = 0.1$ is accepted, and Δ_k is kept unchanged.
True. $\rho_k = 0.3 > \eta$, so accepted. And $1/4 \leq 0.3 \leq 3/4$, so $\Delta_{k+1} = \Delta_k$.
- ③ The TRS solution always lies on the boundary of the trust region.

Check Your Understanding: The Algorithm

True or False?

- ① If $\rho_k = 0.9$ but $\|p_k\| < \Delta_k$, we expand Δ_{k+1} .
False. Expand only if *on the boundary*. If the step is interior, Δ_k isn't constraining progress.
- ② A step with $\rho_k = 0.3$ and $\eta = 0.1$ is accepted, and Δ_k is kept unchanged.
True. $\rho_k = 0.3 > \eta$, so accepted. And $1/4 \leq 0.3 \leq 3/4$, so $\Delta_{k+1} = \Delta_k$.
- ③ The TRS solution always lies on the boundary of the trust region.
False. When the unconstrained minimizer p^N is inside, $p^* = p^N$ with $\lambda^* = 0$.

Check Your Understanding: The Algorithm

True or False?

- ① If $\rho_k = 0.9$ but $\|p_k\| < \Delta_k$, we expand Δ_{k+1} .
False. Expand only if *on the boundary*. If the step is interior, Δ_k isn't constraining progress.
- ② A step with $\rho_k = 0.3$ and $\eta = 0.1$ is accepted, and Δ_k is kept unchanged.
True. $\rho_k = 0.3 > \eta$, so accepted. And $1/4 \leq 0.3 \leq 3/4$, so $\Delta_{k+1} = \Delta_k$.
- ③ The TRS solution always lies on the boundary of the trust region.
False. When the unconstrained minimizer p^N is inside, $p^* = p^N$ with $\lambda^* = 0$.

Short answer: Why does Algorithm 4.1 only expand Δ_k when $\|p_k\| = \Delta_k$?

Check Your Understanding: The Algorithm

True or False?

- ① If $\rho_k = 0.9$ but $\|p_k\| < \Delta_k$, we expand Δ_{k+1} .
False. Expand only if *on the boundary*. If the step is interior, Δ_k isn't constraining progress.
- ② A step with $\rho_k = 0.3$ and $\eta = 0.1$ is accepted, and Δ_k is kept unchanged.
True. $\rho_k = 0.3 > \eta$, so accepted. And $1/4 \leq 0.3 \leq 3/4$, so $\Delta_{k+1} = \Delta_k$.
- ③ The TRS solution always lies on the boundary of the trust region.
False. When the unconstrained minimizer p^N is inside, $p^* = p^N$ with $\lambda^* = 0$.

Short answer: Why does Algorithm 4.1 only expand Δ_k when $\|p_k\| = \Delta_k$?

If the step is interior, the radius isn't the bottleneck — the algorithm already found the unconstrained optimum of m_k . Making Δ_k bigger wouldn't change the next step.

Assumptions for Convergence

We work under the following **standing assumptions**:

Assumptions

- (A1) $f : \mathbb{R}^n \rightarrow \mathbb{R}$ is continuously differentiable.
- (A2) f is bounded below: there exists f_{low} such that $f(x) \geq f_{\text{low}}$ for all x .
- (A3) ∇f is Lipschitz continuous: there exists $L > 0$ such that

$$\|\nabla f(x) - \nabla f(y)\| \leq L\|x - y\| \quad \forall x, y.$$

- (A4) The model Hessians are uniformly bounded: $\|B_k\| \leq \beta$ for all k .
- (A5) Each subproblem solution achieves at least **Cauchy decrease**:

$$m_k(0) - m_k(p_k) \geq \frac{1}{2}\|g_k\| \min\left(\Delta_k, \frac{\|g_k\|}{\|B_k\|}\right)$$

Assumptions (A1)–(A3) are standard. (A4) is mild (BFGS/SR1 with safeguards). (A5) is achieved by any reasonable subproblem solver (Cauchy, dogleg, CG).

Key Lemma: Model Accuracy Controls ρ_k

Lemma (N&W, based on proof of Thm. 4.5)

Under assumptions (A1), (A3), (A4), for any step p_k with $\|p_k\| \leq \Delta_k$:

$$|\rho_k - 1| = \frac{|f(x_k + p_k) - m_k(p_k)|}{|m_k(0) - m_k(p_k)|} \leq \frac{(L + \beta)\Delta_k^2/2}{\text{pred}_k}$$

Proof. By Taylor's theorem and (A3):

$$|f(x_k + p_k) - m_k(p_k)| = \left| \frac{1}{2} p_k^T (\nabla^2 f(\xi) - B_k) p_k \right| \leq \frac{1}{2} (L + \beta) \|p_k\|^2 \leq \frac{1}{2} (L + \beta) \Delta_k^2$$

for some ξ on the segment $[x_k, x_k + p_k]$. Dividing by pred_k gives the result. □

Consequence: When Δ_k is small relative to pred_k , we get $\rho_k \approx 1$ (model is accurate). This is the fundamental mechanism ensuring the algorithm makes progress.

Global Convergence Theorem

Theorem (N&W, Theorem 4.5)

Under assumptions (A1)–(A5), the iterates produced by Algorithm 4.1 satisfy:

$$\liminf_{k \rightarrow \infty} \|\nabla f(x_k)\| = 0$$

Remark: This says we find *approximate* first-order critical points. It does not say every subsequence converges, nor that we find a minimizer — only that the gradient is arbitrarily small infinitely often. (Stronger results hold under additional assumptions.)

Proof of Global Convergence (Part 1)

Proof by contradiction. Suppose $\|g_k\| \geq \epsilon > 0$ for all k .

Step 1: The radius doesn't shrink to zero.

By Cauchy decrease (A5):

$$\text{pred}_k \geq \frac{1}{2}\epsilon \min\left(\Delta_k, \frac{\epsilon}{\beta}\right)$$

From the Key Lemma, if $\Delta_k \leq \frac{\epsilon}{(L+\beta)}$, then:

$$|\rho_k - 1| \leq \frac{(L+\beta)\Delta_k^2/2}{\frac{1}{2}\epsilon\Delta_k} = \frac{(L+\beta)\Delta_k}{\epsilon} \leq 1 \cdot \frac{\epsilon/(L+\beta) \cdot (L+\beta)}{\epsilon} = 1$$

More precisely, if $\Delta_k \leq \frac{\epsilon}{2(L+\beta)}$, then $|\rho_k - 1| \leq \frac{1}{2}$, so $\rho_k \geq \frac{1}{2} > \frac{1}{4}$.

When $\rho_k > \frac{1}{4}$, the algorithm does *not shrink* Δ_k . Since Δ_k is only multiplied by $\frac{1}{4}$ when $\rho_k < \frac{1}{4}$, after finitely many shrinks we reach:

$$\Delta_k \geq \hat{\delta} := \frac{1}{4} \cdot \frac{\epsilon}{2(L+\beta)} > 0 \quad \text{for all large } k.$$

Proof of Global Convergence (Part 2)

Step 2: Successful steps give sufficient decrease.

When $\rho_k > \eta$ (step accepted), by definition of ρ_k :

$$f(x_k) - f(x_{k+1}) = \text{ared}_k = \rho_k \cdot \text{pred}_k \geq \eta \cdot \text{pred}_k$$

By Cauchy decrease and the lower bound on Δ_k :

$$\text{pred}_k \geq \frac{1}{2}\epsilon \min\left(\hat{\delta}, \frac{\epsilon}{\beta}\right) =: c > 0$$

So on every successful step: $f(x_k) - f(x_{k+1}) \geq \eta c > 0$.

Step 3: Contradiction.

There are infinitely many successful steps (otherwise $\Delta_k \rightarrow 0$, contradicting Step 1). Telescoping:

$$f(x_0) - f_{\text{low}} \geq \sum_{\substack{k: \\ \text{successful}}} (f(x_k) - f(x_{k+1})) \geq \sum_{\substack{k: \\ \text{successful}}} \eta c = +\infty$$

This contradicts $f \geq f_{\text{low}}$ (A2). So the assumption $\|g_k\| \geq \epsilon > 0$ for all k must be false. □

Strengthening: $\lim$ vs. $\liminf$

Under slightly stronger conditions, we can upgrade to:

$$\lim_{k \rightarrow \infty} \|\nabla f(x_k)\| = 0$$

Theorem (N&W, Theorem 4.6)

If, in addition to (A1)–(A5), the iterates $\{x_k\}$ remain in a **compact** set (or equivalently, the sublevel set $\{x : f(x) \leq f(x_0)\}$ is compact), then:

$$\lim_{k \rightarrow \infty} \|\nabla f(x_k)\| = 0$$

Proof idea. Compactness ensures that every subsequence has a convergent sub-subsequence. The argument from Theorem 4.5 applies to each, giving a critical point as an accumulation point. A continuity argument then shows the gradient converges to zero along the *full* sequence.

Practical note: Compactness of the sublevel set holds whenever $f(x) \rightarrow \infty$ as $\|x\| \rightarrow \infty$ (coercivity), which is common in applications.

Local Convergence: Setup

Near a strict local minimizer x^* with $\nabla^2 f(x^*) \succ 0$:

- The Newton step $p_k^N = -[\nabla^2 f(x_k)]^{-1} g_k$ satisfies $\|p_k^N\| \rightarrow 0$ as $x_k \rightarrow x^*$.
- For k large enough, $\|p_k^N\| < \Delta_k$, so the trust region constraint is **inactive**.
- The trust region method reduces to unconstrained Newton (or quasi-Newton).

Theorem (Eventual Acceptance of Full Newton Step)

Suppose $\nabla^2 f$ is Lipschitz continuous near x^* , $\nabla^2 f(x^*) \succ 0$, and $B_k = \nabla^2 f(x_k)$. Then for all k sufficiently large:

- 1 $\|p_k^N\| < \Delta_k$ (Newton step is inside the trust region).
- 2 $\rho_k \rightarrow 1$ (model becomes increasingly accurate).
- 3 The step is accepted: $x_{k+1} = x_k + p_k^N$.

Hence the trust region method achieves **quadratic convergence**:

$$\|x_{k+1} - x^*\| = O(\|x_k - x^*\|^2)$$

Proof Sketch: Eventual Newton Steps

Why $\rho_k \rightarrow 1$: By Taylor expansion with Lipschitz Hessian:

$$|f(x_k + p_k^N) - m_k(p_k^N)| = O(\|p_k^N\|^3)$$

while $\text{pred}_k = O(\|p_k^N\|^2)$ (since $p_k^N = -H_k^{-1}g_k$ and $g_k = O(\|x_k - x^*\|)$).

Therefore $|\rho_k - 1| = O(\|p_k^N\|) \rightarrow 0$.

Why Δ_k stays large enough: Since $\rho_k > 3/4$ eventually, the algorithm *expands* Δ_k whenever $\|p_k\| = \Delta_k$. Once the Newton step is accepted with $\|p_k^N\| < \Delta_k$, the radius is maintained or increased.

Quadratic convergence then follows from standard Newton theory:

$$\|x_{k+1} - x^*\| = \|x_k + p_k^N - x^*\| \leq \frac{L_H}{2\lambda_{\min}(\nabla^2 f(x^*))} \|x_k - x^*\|^2$$

where L_H is the Lipschitz constant of the Hessian.

Convergence Rate Summary

Subproblem solver	Local rate	Condition
Cauchy point only	Linear	Always
Dogleg	Quadratic	$B_k = \nabla^2 f(x_k)$, $B_k \succ 0$
Steihaug-Toint CG	Superlinear	$B_k = \nabla^2 f(x_k)$
Exact TRS solve	Quadratic	$B_k = \nabla^2 f(x_k)$
BFGS (quasi-Newton)	Superlinear	Dennis–Moré condition
SR1 (quasi-Newton)	Superlinear	Bounded approx. error

Dennis–Moré Condition for Superlinear Convergence

A quasi-Newton method converges superlinearly if and only if:

$$\lim_{k \rightarrow \infty} \frac{\|(B_k - \nabla^2 f(x^*))(x_k - x^*)\|}{\|x_k - x^*\|} = 0$$

BFGS satisfies this under standard assumptions. SR1 satisfies it under a bounded deterioration condition.

Check Your Understanding: Convergence

True or False?

- 1 “Global convergence” of trust region methods means the iterates converge to a *global* minimizer of f .

Check Your Understanding: Convergence

True or False?

- ① “Global convergence” of trust region methods means the iterates converge to a *global* minimizer of f .
False. It means gradients converge to zero: $\|g_k\| \rightarrow 0$. This gives convergence to a *stationary point*, which could be a local min, saddle, or even a local max.
- ② If we always use the Cauchy point (never dogleg or CG), the algorithm still converges globally.

Check Your Understanding: Convergence

True or False?

- ① “Global convergence” of trust region methods means the iterates converge to a *global* minimizer of f .
False. It means gradients converge to zero: $\|g_k\| \rightarrow 0$. This gives convergence to a *stationary point*, which could be a local min, saddle, or even a local max.
- ② If we always use the Cauchy point (never dogleg or CG), the algorithm still converges globally.
True. The Cauchy decrease bound is exactly what the convergence proof needs. Better methods converge *faster*, but aren't needed for the guarantee.

Check Your Understanding: Convergence

True or False?

- ① “Global convergence” of trust region methods means the iterates converge to a *global* minimizer of f .

False. It means gradients converge to zero: $\|g_k\| \rightarrow 0$. This gives convergence to a *stationary point*, which could be a local min, saddle, or even a local max.

- ② If we always use the Cauchy point (never dogleg or CG), the algorithm still converges globally.

True. The Cauchy decrease bound is exactly what the convergence proof needs. Better methods converge *faster*, but aren't needed for the guarantee.

Short answer: Near a strong minimizer ($\nabla^2 f(x^*) \succ 0$), the trust region Newton method eventually takes pure Newton steps. Why does the TR constraint become inactive?

Check Your Understanding: Convergence

True or False?

- ① “Global convergence” of trust region methods means the iterates converge to a *global* minimizer of f .

False. It means gradients converge to zero: $\|g_k\| \rightarrow 0$. This gives convergence to a *stationary point*, which could be a local min, saddle, or even a local max.

- ② If we always use the Cauchy point (never dogleg or CG), the algorithm still converges globally.

True. The Cauchy decrease bound is exactly what the convergence proof needs. Better methods converge *faster*, but aren't needed for the guarantee.

Short answer: Near a strong minimizer ($\nabla^2 f(x^*) \succ 0$), the trust region Newton method eventually takes pure Newton steps. Why does the TR constraint become inactive?

Two things happen simultaneously: (1) $\|p_k^N\| \rightarrow 0$ as $x_k \rightarrow x^*$, and (2) Δ_k stays bounded away from zero because $\rho_k \rightarrow 1$ (the quadratic model becomes exact near the minimum). So eventually $\|p_k^N\| \ll \Delta_k$.

Iteration Complexity Bounds

Question: How many iterations to find x_k with $\|g_k\| \leq \epsilon$?

Theorem (Gratton, Sartenaer, Toint (2008); Cartis, Gould, Toint (2011))

Under (A1)–(A5), the trust region method with Cauchy decrease finds an iterate with $\|\nabla f(x_k)\| \leq \epsilon$ in at most

$$O\left(\frac{f(x_0) - f_{\text{low}}}{\epsilon^2}\right) \text{ iterations.}$$

Proof sketch. On each successful iteration, f decreases by at least $c \in \Delta_k$. Since $\Delta_k \geq \hat{\delta} = \Theta(\epsilon)$ (from the convergence proof), each successful step reduces f by $\Omega(\epsilon^2)$. Total decrease is bounded by $f(x_0) - f_{\text{low}}$, giving at most $O(\epsilon^{-2})$ successful steps. The number of unsuccessful steps is bounded by a constant times the number of successful ones (geometric shrinking argument).

Same rate as gradient descent — but with better constants and robustness.

Second-Order Complexity

For second-order methods (exact or near-exact TRS solves with $B_k = \nabla^2 f(x_k)$):

Theorem (Cartis, Gould, Toint (2012))

With exact Hessian and exact subproblem solves, a trust region method finds x_k with $\|\nabla f(x_k)\| \leq \epsilon$ in at most $O(\epsilon^{-3/2})$ iterations.

This matches the complexity of **cubic regularization** (Nesterov–Polyak, 2006).

Method	Complexity to $\ g\ \leq \epsilon$	Order info
Gradient descent / Cauchy TR	$O(\epsilon^{-2})$	1st order
Newton TR / ARC	$O(\epsilon^{-3/2})$	2nd order
Cubic regularization	$O(\epsilon^{-3/2})$	2nd order

The $O(\epsilon^{-3/2})$ bound is **optimal** among second-order methods with Lipschitz Hessians (lower bound by Cartis et al.).

Cubic Regularization: The Dual Framework

Adaptive Regularization with Cubics (ARC)

Instead of constraining step size, penalize with a cubic:

$$\min_p f(x_k) + g_k^T p + \frac{1}{2} p^T \nabla^2 f(x_k) p + \frac{\sigma_k}{3} \|p\|^3$$

Adapt σ_k (like Δ_k) based on the ratio ρ_k .

	Trust Region	ARC
Constraint	$\ p\ \leq \Delta_k$	None
Penalty	None	$\frac{\sigma_k}{3} \ p\ ^3$
Adaptive param	Δ_k (radius)	σ_k (regularization)
1st-order complexity	$O(\epsilon^{-2})$	$O(\epsilon^{-3/2})$
2nd-order complexity	$O(\epsilon^{-3/2})$	$O(\epsilon^{-3/2})$

ARC achieves $O(\epsilon^{-3/2})$ even with *first-order* subproblem solves — a genuine advantage over classical TR.

Application 1: Nonlinear Least Squares

For $f(x) = \frac{1}{2}\|r(x)\|^2$ with residual $r : \mathbb{R}^n \rightarrow \mathbb{R}^m$, the exact Hessian is $\nabla^2 f = J^T J + \sum_i r_i \nabla^2 r_i$. The Gauss–Newton approximation drops the second-order term:

$$B_k = J_k^T J_k \succeq 0 \quad \implies \quad m_k(p) = \frac{1}{2}\|r_k + J_k p\|^2 \quad \text{s.t.} \quad \|p\| \leq \Delta_k$$

The optimality conditions give: $(J_k^T J_k + \lambda I)p = -J_k^T r_k$. This is the **Levenberg–Marquardt method** — historically the *first* trust region method (Levenberg 1944, Marquardt 1963)!

Why it works so well: $B_k = J^T J$ is always PSD, so the hard case never arises and the dogleg method applies. Near a zero-residual solution ($\|r^*\| \approx 0$), $J^T J$ approximates the Hessian well, giving superlinear convergence. The regularization λI handles rank-deficient Jacobians gracefully.

Where Levenberg–Marquardt is Used

Curve fitting and sensor calibration (the original motivation), robotics (inverse kinematics, SLAM), computer vision (bundle adjustment — Ceres Solver), neural network training (small/medium), and chemical engineering (kinetic parameter estimation).

Application 2: Trust Regions in Machine Learning

Trust Region Policy Optimization (TRPO) — Schulman et al., 2015

In reinforcement learning, update policy π_θ by:

$$\max_{\theta} \hat{\mathbb{E}} \left[\frac{\pi_\theta(a|s)}{\pi_{\theta_{\text{old}}}(a|s)} \hat{A}(s, a) \right] \quad \text{s.t.} \quad \hat{\mathbb{E}}[D_{\text{KL}}(\pi_{\theta_{\text{old}}} \| \pi_\theta)] \leq \delta$$

The trust region is in distribution space, not parameter space!

- KL divergence replaces $\|p\|_2 \leq \Delta$.
- Ensures policy updates don't change behavior too drastically.
- **PPO** replaces the hard constraint with a clipped penalty — analogous to the regularization dual view.

Connection to theory: TRPO's performance guarantee is a direct analogue of our ρ_k analysis — model fidelity (surrogate objective accuracy) controls step acceptance.

Application 3: Proximal and Mirror Methods

Many modern algorithms are trust region methods in disguise:

Proximal Point Algorithm

$$x_{k+1} = \arg \min_x f(x) + \frac{1}{2\eta} \|x - x_k\|^2$$

Trust region with implicit radius determined by η . Model = f itself (not quadratic).

Proximal gradient (composite optimization): For $\min g(x) + h(x)$ (g smooth, h nonsmooth):

$$x_{k+1} = \text{prox}_{\eta h}(x_k - \eta \nabla g(x_k))$$

Trust region on the smooth part, exact step on the nonsmooth part.

Mirror descent: Replace $\frac{1}{2\eta} \|x - x_k\|^2$ with a *Bregman divergence* $D_\phi(x, x_k)$:

$$x_{k+1} = \arg \min_x \langle \nabla f(x_k), x \rangle + \frac{1}{\eta} D_\phi(x, x_k)$$

Trust region w.r.t. the geometry induced by ϕ .

Application 4: Generalized Trust Regions

Trust Region Shape	Metric	Application
$\ p\ _2 \leq \Delta$	Euclidean	Standard (N&W Ch. 4)
$\ p\ _\infty \leq \Delta$	ℓ_∞	Box constraints
$p^T H p \leq \Delta^2$	Ellipsoidal	Preconditioning
$D_{\text{KL}}(\cdot \ \cdot) \leq \delta$	KL divergence	TRPO, natural gradient
$D_\phi(\cdot, \cdot) \leq \delta$	Bregman	Mirror descent
$W_2(\cdot, \cdot) \leq \delta$	Wasserstein	Distributionally robust opt.
Tangent space of $\mathcal{M}$	Riemannian	Manifold optimization

Unifying principle: A trust region method is any algorithm that restricts iterates to a neighborhood — measured by some notion of distance or divergence — where the model is reliable.

The convergence theory generalizes: as long as you have Cauchy-type decrease and adaptive radius/divergence control, global convergence follows.

Application 5: Optimization on Manifolds

For $\min_{x \in \mathcal{M}} f(x)$ where $\mathcal{M}$ is a Riemannian manifold:

The TRS is posed in the **tangent space** $T_x \mathcal{M}$:

$$\min_{\eta \in T_x \mathcal{M}} \langle \text{grad } f(x), \eta \rangle_x + \frac{1}{2} \langle \text{Hess } f(x)[\eta], \eta \rangle_x \quad \text{s.t.} \quad \|\eta\|_x \leq \Delta$$

then retract: $x_{k+1} = R_{x_k}(\eta_k)$.

Applications

- **Low-rank matrix/tensor completion:** Fixed-rank manifold
- **PCA, subspace tracking:** Stiefel or Grassmann manifold
- **Rotation averaging (computer vision):** $SO(3)$ manifold
- **Covariance estimation:** Symmetric positive definite (SPD) manifold

Software: Manopt, Pymanopt, Manifolds.jl

Global convergence of Riemannian trust region methods follows analogously (Absil, Mahony, Sepulchre 2008).

Application 6: Large-Scale and Modern Solvers

When n is very large, we cannot form B_k explicitly.

Matrix-Free Approaches

- **Steihaug-Toint CG:** Hessian-vector products via automatic differentiation at cost $\approx 4\times$ a gradient evaluation.
- **GLTR (Gould, Lucidi, Roma, Toint):** Lanczos-based; projects TRS onto Krylov subspace.
- **Randomized subspace methods:** Sketch the TRS to a low-dimensional subspace using random projections, solve exactly there.

Software ecosystem:

- PETSc/TAO: Large-scale TR with CG subsolves (C/Fortran/Python)
- Ceres Solver: TR for NLS, bundle adjustment (C++)
- GALAHAD: Full TR/ARC suite (Fortran)
- `scipy.optimize.minimize(method='trust-krylov')`: Matrix-free TR (Python)

Case Study: Trust-Region Neural MHE for Quadrotors

Paper

B. Wang, Z. Ma, S. Lai, and L. Zhao. “Trust-Region Neural Moving Horizon Estimation for Robots.” *IEEE Robotics and Automation Letters / ICRA*, 2024.

Code: github.com/BinghengNUS/TR-NeuroMHE

Problem: A quadrotor drone flying aggressively (speeds >12 m/s, accelerations $>3g$) experiences complex **aerodynamic disturbances** — forces from drag, blade-vortex interactions, and ground effects that are hard to model analytically.

Goal: Estimate these unknown disturbance forces $d_f \in \mathbb{R}^3$ in real time so the flight controller can compensate.

Why it matters: Without accurate disturbance estimation, the drone’s tracking error increases dramatically during aggressive maneuvers — potentially causing crashes.

Where trust regions enter: The estimation problem has a *bilevel* structure. The inner level solves an optimal estimation problem (MHE). The outer level tunes the estimator’s parameters using a trust region method.

The Optimization Problem

Moving Horizon Estimation (MHE) estimates the drone's state by solving:

$$\hat{x} = \arg \min_{x_{0:N}} \underbrace{\|x_0 - \bar{x}_0\|_{P^{-1}}^2}_{\text{prior}} + \sum_{i=0}^{N-1} \underbrace{\|w_i\|_{Q^{-1}(\theta)}^2}_{\text{process noise}} + \sum_{i=0}^N \underbrace{\|v_i\|_{R^{-1}(\theta)}^2}_{\text{measurement noise}}$$

subject to the quadrotor dynamics $x_{i+1} = f(x_i, u_i) + w_i$.

The weighting matrices $Q(\theta), R(\theta)$ are parameterized by a small **neural network** with parameters θ (“NeuroMHE”).

Outer (learning) problem: Find θ that minimizes estimation error:

$$\min_{\theta} L(\hat{x}(\theta)) = \sum_t \|d_{f,t}^{\text{true}} - \hat{d}_{f,t}(\theta)\|^2$$

This is a **nonconvex, nonlinear** optimization over the neural network parameters θ .

Why Trust Regions? (Not SGD, Not Adam)

Challenges with gradient descent:

- The loss landscape is **highly nonconvex** (neural network + MHE composition).
- **Step size sensitivity**: Too large $\Rightarrow$ divergence. Too small $\Rightarrow$ slow convergence.
- Only **100 training data points** available (real flight data is expensive).
- Need **robustness to initialization** — different random seeds should give similar results.

Trust region advantages here:

- **Adaptive step size** via ρ_k — automatically handles varying curvature.
- **Second-order convergence** — much faster with only 100 data points.
- **Robust to initialization** — does not overshoot.
- Hessian H_k is available **cheaply** via a Kalman filter recursion (key insight of the paper).

The trust region subproblem at each training step is exactly our TRS:

$$\min_{\mu} \nabla L(\theta_k)^T \mu + \frac{1}{2} \mu^T H_k \mu \quad \text{s.t.} \quad \|\mu\| \leq \Delta_k$$

Update: $\theta_{k+1} = \theta_k + \mu^*$ if $\rho_k > \eta$.

The Key Innovation: Cheap Hessians via Kalman Filter

Computing the Hessian $H_k = \nabla_{\theta}^2 L$ naively would require differentiating through the entire MHE solve — expensive.

Key insight: The MHE gradient $\nabla_{\theta} L$ is already computed recursively via a **Kalman filter** (from the KKT conditions of the MHE). The authors show that *the same Kalman filter* can be reused to compute the Hessian with only minor input modifications:

	Gradient	Hessian
Method	Kalman filter on KKT	Same Kalman filter, modified inputs
Complexity	$O(N)$ per horizon	$O(N)$ per horizon
Extra cost	—	$\sim 2\times$ gradient cost

This is analogous to how **automatic differentiation** gives Hessian-vector products at $\sim 4\times$ gradient cost — but here it's done analytically through the estimation structure.

Results: Trust Region vs. Gradient Descent

Evaluated on 13 real agile flight trajectories (unseen during training):

Method	Force RMSE	Training time	Parameters
NeuroBEM (state-of-art)	0.79 N	hours	31,681
NeuroMHE (grad. descent)	0.42 N	12 min	446
TR-NeuroMHE (trust region)	0.25 N	<5 min	446

Key numbers:

- **68.1% improvement** in force estimation over state-of-the-art.
- Uses only **1.4%** of NeuroBEM's parameters.
- Trains in **<5 minutes** with only **100 data points**.
- Trust region method converges in **fewer episodes** than gradient descent, and the total training time is shorter despite computing Hessians.

Why Trust Regions Won: Connecting to Theory

- ➊ **Adaptive Δ_k :** When the neural network's loss landscape has sharp curvature changes (common in nonconvex problems), the trust region radius *automatically shrinks* ($\rho_k < 1/4$), preventing the divergence that gradient descent with fixed step size suffers from.
- ➋ **Quadratic convergence near optimum:** With exact Hessians and the solution inside the trust region ($\|p^N\| < \Delta_k$), the method reduces to Newton's method — converging quadratically in the final phase. This is why it needs fewer iterations.
- ➌ **Robust to initialization:** Theorem 4.5 guarantees $\liminf \|g_k\| = 0$ regardless of starting point. In practice, the TR method found similar solutions from **all random seeds tested**, while gradient descent showed high variance.
- ➍ **Small data regime:** With only 100 training points, every iteration counts. Second-order methods extract maximal information per step — the Hessian captures curvature that the gradient alone misses.

Takeaway: The classical N&W trust region framework (Algorithm 4.1, Theorems 4.1 and 4.5) directly explains *why* this robotics method works.

Case Study: Trust Regions for Chained Stewart Platforms

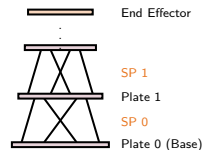
Problem: Move a stack of four chained Stewart platforms (“Assembler”) from pose A to pose B while **minimizing worst-case leg forces** throughout the motion.

- 24 DOF, over-actuated parallel mechanism
- Fixed base plate, target end-effector pose
- Internal plate poses can be chosen freely $\Rightarrow$ optimization opportunity

Challenge: Highly nonconvex feasible space

- Trigonometric, bilinear, quadratic constraints
- Many local optima (22 found from 100 random starts on a 2-SP instance)
- Naïve interpolation: only $\sim 1\%$ behaved paths

Ref: Beach, Chapin, Glassner, Hildebrand, Komendera — Virginia Tech.



Stewart Platform: Trust Region Formulation

At each step along the trajectory, solve a **full NLP** with motion-limiting constraints that act as a trust region:

Trust region constraints (per plate i):

$$\|\mathbf{p}_i - \mathbf{p}_i^{\text{init}}\| \leq \varepsilon^{\text{pos}}, \quad \|R_i - R_i^{\text{init}}\|_F \leq \varepsilon^{\text{rot}}$$

These play the role of $\|p\| \leq \Delta_k$ — restricting each step to a region where the previous configuration is still “trustworthy.”

Objective (balancing forces vs. progress toward goal):

$$\min \lambda^{\text{force}} \left(\tau^{\text{ends}} + \frac{\lambda^{\text{avg}}}{6N_P} \tau^{1\text{-norm}} + 1000 \tau^{\text{viol}} \right) + \lambda^{\text{pose}} \left(E^{\text{pos}} + \frac{1}{4} E^{\text{rot}} \right)$$

- τ^{ends} : max force above endpoint forces (keep forces bounded)
- τ^{viol} : max force above f^{max} (penalize constraint violation heavily)
- $E^{\text{pos}}, E^{\text{rot}}$: distance to goal pose

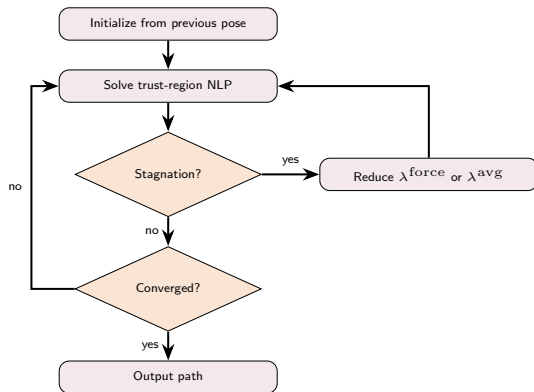
Plus all kinematic validity constraints (leg lengths, joint angles, collision avoidance) and force definitions via spatial Jacobian.

Stewart Platform: Iteration and Adaptation

Algorithm outline:

- 1 Optimize start and goal poses (single-pose NLP)
- 2 Record $f^{\max, \text{ends}}$ from both endpoints
- 3 **Repeat:**
 - Solve trust-region NLP from current pose
 - If **stagnation**: reduce $\lambda^{\text{force}} \leftarrow \lambda^{\text{force}}/2$
 - If **wrong direction**: reduce $\lambda^{\text{avg}} \leftarrow \lambda^{\text{avg}}/4$
 - Otherwise: accept step, advance
- 4 Converge when all plates within $(\varepsilon^{\text{pos}}, \varepsilon^{\text{rot}})$ of goal

Backtracking: If convergence fails, restart with $\lambda^{\text{force}} \leftarrow \lambda^{\text{force}}/4$ (up to 5 times).



Stewart Platform: Results and Theory Connection

Dataset	% Behaved	% Force-Valid
<i>Trust Region Method</i>		
Uniform	100%	100%
Extreme	99.9%	100%
Repeated	99.8%	100%
<i>Naïve Interpolation</i>		
Uniform	1.0%	78.5%
Extreme	1.3%	75.1%
Repeated	0.3%	19.7%

Behaved: mid-path forces $\leq$ endpoint forces.

Force-valid: all forces within hardware limits.

Connection to N&W theory:

- Motion-limiting constraints $\|p - p^{\text{init}}\| \leq \varepsilon$ play the role of the trust region ball $\|p\| \leq \Delta_k$
- Adaptive λ weighting with backtracking substitutes for the ρ_k -based radius update
- Over-actuation $\Rightarrow$ many local optima; trust region prevents jumping between distant solutions

What differs from Ch. 4:

- Full NLP solve (IPOPT) at each step, not a quadratic model
- Fixed radii $\varepsilon^{\text{pos}}, \varepsilon^{\text{rot}}$ (not adaptive)
- Adaptation via objective weights, not radius

Result: $\sim 100\%$ force-valid trajectories vs. $< 2\%$ for naïve interpolation.

Application: Robot Inverse Kinematics

Problem: Given a desired end-effector position $x_{\text{target}} \in \mathbb{R}^3$, find joint angles $\theta \in \mathbb{R}^n$ so the robot's hand reaches the target.

A robot arm with n joints has a **forward kinematics** map:

$$x(\theta) = f_{\text{FK}}(\theta) \in \mathbb{R}^3$$

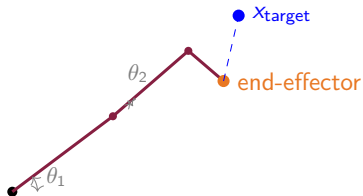
Inverse kinematics (IK): solve the nonlinear system

$$f_{\text{FK}}(\theta) = x_{\text{target}}$$

or equivalently, minimize the residual:

$$\min_{\theta} \frac{1}{2} \|f_{\text{FK}}(\theta) - x_{\text{target}}\|^2$$

This is a **nonlinear least-squares** problem — exactly the setting for Levenberg–Marquardt!



Inverse Kinematics: The Trust Region Connection

The Jacobian of the forward kinematics is:

$$J(\theta) = \frac{\partial f_{\text{FK}}}{\partial \theta} \in \mathbb{R}^{3 \times n}$$

Gauss–Newton model: Linearize $f_{\text{FK}}(\theta + p) \approx f_{\text{FK}}(\theta) + J(\theta)p$:

$$\min_p \frac{1}{2} \|r(\theta) + J(\theta)p\|^2 \quad \text{s.t.} \quad \|p\| \leq \Delta$$

where $r(\theta) = f_{\text{FK}}(\theta) - x_{\text{target}}$.

This is the **trust region subproblem** with $B_k = J^T J$ and $g_k = J^T r$. The normal equations give the **Levenberg–Marquardt** step:

$$(J^T J + \lambda I) p = -J^T r$$

Why trust regions matter here:

- Near **singularities** (arm fully extended), J becomes rank-deficient $\Rightarrow J^T J$ is singular. The damping λI regularizes it — same as the TRS multiplier!
- **Multiple solutions** exist (“elbow up” vs “elbow down”); the trust region keeps iterates near the current configuration.

Ref: Sugihara (2011), IEEE Trans. Robotics. Buss (2004), IK survey. MuJoCo, Pinocchio use LM internally.

Application: Bundle Adjustment in 3D Reconstruction

Problem: Given m photos of a scene with n tracked feature points, jointly optimize all **camera poses** $\{C_i\}_{i=1}^m$ and **3D point positions** $\{X_j\}_{j=1}^n$ to minimize **reprojection error**.

$$\min_{\{C_i\}, \{X_j\}} \sum_{(i,j) \in \mathcal{O}} \left\| \underbrace{\pi(C_i, X_j)}_{\text{predicted pixel}} - \underbrace{u_{ij}}_{\text{observed pixel}} \right\|^2$$

where $\pi(C_i, X_j)$ projects 3D point X_j onto camera i 's image plane.

Scale: Typical problems have $m \sim 10^2$ – 10^4 cameras, $n \sim 10^4$ – 10^6 points.

- Parameters: $6m + 3n$ (6 DOF per camera + 3 per point) $\sim 10^4$ – 10^6
- Residuals: $2|\mathcal{O}|$ (2 per observation) $\sim 10^5$ – 10^7

Again a **nonlinear least-squares** problem $\Rightarrow$ Levenberg–Marquardt / trust region.

Ref: Triggs et al. (2000), “Bundle Adjustment — A Modern Synthesis.” Agarwal et al., Ceres Solver.

Bundle Adjustment: Exploiting Sparsity

The Jacobian $J \in \mathbb{R}^{2|\mathcal{O}| \times (6m+3n)}$ is extremely **sparse**: each residual depends on only 1 camera (6 params) and 1 point (3 params).

The LM normal equations $(J^T J + \lambda I)p = -J^T r$ have a special **block structure**:

$$\begin{pmatrix} U + \lambda I & W \\ W^T & V + \lambda I \end{pmatrix} \begin{pmatrix} \delta C \\ \delta X \end{pmatrix} = - \begin{pmatrix} \epsilon_C \\ \epsilon_X \end{pmatrix}$$

where U is block-diagonal (m blocks of 6×6) and V is block-diagonal (n blocks of 3×3).

Schur complement trick: Eliminate δX cheaply (since $V + \lambda I$ is block-diagonal):

$$\underbrace{(U + \lambda I - W(V + \lambda I)^{-1}W^T)}_{\text{reduced camera system, } 6m \times 6m} \delta C = -(\epsilon_C - W(V + \lambda I)^{-1}\epsilon_X)$$

This reduces a $10^6 \times 10^6$ system to a $10^3 \times 10^3$ system!

Software: Ceres Solver (Google, C++) implements LM + Dogleg with Schur complement solvers, iterative Schur with preconditioners, GPU acceleration. Used in Google Maps, Street View, and most SfM pipelines.

Application: Radiation Therapy Treatment Planning

Problem: Design radiation beam intensities to deliver a prescribed dose to a tumor while minimizing damage to surrounding healthy tissue.

Decision variables: Beam intensities $w \in \mathbb{R}_+^n$ ($n \sim 10^3$ – 10^4 beamlets).

Dose model: The dose at voxel i is $d_i(w) = \sum_j A_{ij} w_j$, where $A \in \mathbb{R}^{m \times n}$ is the dose-influence matrix ($m \sim 10^5$ – 10^6 voxels).

Objective: Minimize a weighted penalty:

$$\min_{w \geq 0} \sum_{i \in \text{tumor}} \alpha_i (d_i - D_{\text{rx}})^2 + \sum_{i \in \text{OAR}} \beta_i \max(0, d_i - D_{\text{max}})^2$$

subject to dose-volume constraints (e.g., “ $\leq 5\%$ of the heart receives > 40 Gy”).

This is a **large-scale constrained NLP** with a nonsmooth objective (due to max) and nonconvex dose-volume constraints.

Radiation Therapy: The Trust Region Connection

Solution approach: Interior point or SQP methods, both using **trust region** globalization.

At each iteration, form a quadratic model of the Lagrangian and solve:

$$\min_p \nabla \mathcal{L}_k^T p + \frac{1}{2} p^T H_k p + \text{linearized constraints} \quad \text{s.t.} \quad \|p\| \leq \Delta_k$$

This is a **trust-region SQP** subproblem. The Hessian H_k may be indefinite (nonconvex constraints), so the PSD condition from Theorem 4.1 is essential.

Why trust regions are critical:

- **Patient safety:** Cannot accept a step that increases dose to critical organs. The ρ_k mechanism rejects bad steps before they become treatment plans.
- **Nonconvexity:** Dose-volume constraints create many local minima; trust regions provide robust globalization.
- **Scale:** $\sim 10^6$ voxels $\times 10^4$ beamlets; Steihaug-CG solves the TRS matrix-free.

Ref: Unkelbach et al. (2015), Phys. Med. Biol. IPOPT (Wächter & Biegler, 2006) uses TR internally. Eclipse, RayStation use trust-region-based optimizers.

From N&W Chapter 4 to Training ChatGPT

Trust Region Policy Optimization (Schulman et al., ICML 2015; ~8000 citations):

In reinforcement learning, a *policy* $\pi_\theta(a|s)$ maps states to actions. Update rule:

$$\max_{\theta} \mathbb{E} \left[\frac{\pi_\theta(a|s)}{\pi_{\theta_{\text{old}}}(a|s)} \hat{A}(s, a) \right] \quad \text{s.t.} \quad \overline{D}_{\text{KL}}(\pi_{\theta_{\text{old}}}, \pi_\theta) \leq \delta$$

The **KL divergence** $D_{\text{KL}}(p||q) = \sum_a p(a) \log \frac{p(a)}{q(a)}$ measures how different two probability distributions are. It plays the role of $\|p\|^2$ in the TRS: bounding $D_{\text{KL}} \leq \delta$ keeps the new policy “close” to the old one, just as $\|p\| \leq \Delta_k$ keeps the step close to the current iterate.

N&W Chapter 4	TRPO
$m_k(p) = f_k + g_k^T p + \frac{1}{2} p^T B_k p$	Surrogate advantage objective
$\ p\ \leq \Delta_k$	$D_{\text{KL}} \leq \delta$
Steihaug-Toint CG	Conjugate gradient on $F^{-1}g$
ρ_k ratio test	Backtracking line search

Proximal Policy Optimization (PPO, Schulman 2017) simplified TRPO using a clipped surrogate — and became **the algorithm behind RLHF** for GPT, Claude, Gemini, and DeepSeek.

The punchline: The theory from this lecture — trust regions, CG for the subproblem, monotonic improvement guarantees — is the mathematical foundation for training modern LLMs.

Note: The Fisher information matrix $F = \mathbb{E}[\nabla \log \pi \nabla \log \pi^T]$ plays the role of B_k in the quadratic approximation to D_{KL} . Inverting F gives the *natural gradient* direction — so TRPO's CG solve is exactly computing a natural gradient step within a trust region. Without the trust region, large policy updates can cause catastrophic performance collapse: the policy becomes deterministic, loses exploration ability, and performance drops to zero irreversibly.

Google Ceres Solver: Trust Regions at Scale

Bundle adjustment refines 3D reconstructions from multiple photographs. The name comes from the “bundle” of light rays connecting each 3D point to the cameras that see it. Given m cameras (with unknown poses C_i) and n 3D points (X_j), minimize the **reprojection error** — the mismatch between where a point *should* appear in each image ($\pi(C_i, X_j)$) and where it *actually* appears (u_{ij}):

$$\min_{\{C_i\}, \{X_j\}} \sum_{(i,j) \in \mathcal{O}} \|\pi(C_i, X_j) - u_{ij}\|^2$$

This is a **nonlinear least-squares** problem. **Google’s Ceres Solver** (open-source, used in Google Maps, Street View, COLMAP) solves it via:

- Trust-region Levenberg-Marquardt: $(J^T J + \lambda I)p = -J^T r$ (= TRS with $B = J^T J$)
- **Schur complement** exploits sparse structure: each residual depends on only 1 camera + 1 point, so $J^T J$ is block-sparse. Eliminate the point variables first, solve a small camera-only system.
- Reduces $66,000 \times 66,000$ system to 96×96 (16 cameras $\times$ 6 params each)

Real Ceres output (16 cameras, 22,106 points, 83,718 residuals):

iter	cost	gradient	step	ρ_k
0	4.19e+06	2.16e+07	—	—
1	1.98e+05	5.34e+06	2400	0.96
2	5.09e+04	2.11e+06	1010	0.82
5	1.80e+04	311	10.2	1.00

232× cost reduction in 5 iterations. Note $\rho_k \rightarrow 1$ and $|\text{step}| \rightarrow 0$: classic TR Newton convergence.

Scale in practice: Google Street View solves BA problems with $\sim 100\text{K}$ cameras and millions of 3D points. Autonomous driving pipelines (Waymo, Tesla) run BA in real time. The Schur complement + trust-region LM combination is what makes this computationally feasible — without the sparse structure exploitation, these problems would be intractable.

Recent and Emerging Applications

Trust region methods appear across engineering and science:

Power systems. Optimal power flow (OPF) determines generator outputs to minimize cost while satisfying network constraints (Kirchhoff's laws, voltage limits, line capacities). On heavily loaded grids with 1000+ buses, standard interior-point methods can fail to converge from arbitrary starting points. Trust-region interior-point methods guarantee convergence by rejecting steps that violate the model — critical for real-time grid operations. (Sousa & Torres, IEEE Trans. Power Syst., 2011; Zarrin et al., 2025)

Derivative-free optimization. When gradients are unavailable (black-box simulations, physical experiments, legacy codes), interpolation-based quadratic models built from function evaluations replace ∇f and $\nabla^2 f$ inside a trust-region framework. The trust region controls model validity: the interpolation is only accurate near the current point. Recent work achieves $O(|\log \epsilon| \epsilon^{-2})$ evaluation complexity for nonsmooth composite problems. (Liu, Lam & Roberts, IMA J. Numer. Anal., 2025; Xie & Yuan, 2025)

RL frontiers. Elastic trust regions (ETR) adapt the KL constraint δ per-sample based on advantage signal strength: high-reward samples get larger trust regions (learn more aggressively from rare successes), low-signal samples get tighter constraints (avoid overfitting to noise). Improves reasoning model training on olympiad-level math (AIME 2024/2025 benchmarks). (ETR, arXiv Jan 2026; Shimizu et al., ICONIP 2024)

Summary

- ❶ **Core idea:** Build a local model, restrict optimization to a region where it's trustworthy, adapt the region based on model fidelity (ρ_k).
- ❷ **TRS optimality:** Characterized by $(B + \lambda I)p = -g$ with $\lambda \geq 0$, $B + \lambda I \succeq 0$, and complementarity.
- ❸ **Global convergence:** $\liminf \|g_k\| = 0$ under Cauchy decrease + bounded below + Lipschitz gradient. Proof via contradiction: sufficient decrease per step $\Rightarrow$ infinite total decrease.
- ❹ **Local rates:** Trust region eventually becomes inactive $\Rightarrow$ full Newton/quasi-Newton rates (quadratic/superlinear).
- ❺ **Complexity:** $O(\epsilon^{-2})$ first-order; $O(\epsilon^{-3/2})$ second-order (optimal).
- ❻ **Modern lens:** Trust regions unify regularization, proximal methods, TRPO, natural gradient, and manifold optimization through the principle of *controlled model fidelity*.

References

- [1] J. Nocedal and S.J. Wright. *Numerical Optimization*, 2nd ed. Springer, 2006. Chapter 4.
- [2] A.R. Conn, N.I.M. Gould, Ph.L. Toint. *Trust-Region Methods*. SIAM, 2000.
- [3] C. Cartis, N.I.M. Gould, Ph.L. Toint. Adaptive cubic regularisation methods. *Math. Program.*, 2011.
- [4] S. Gratton, A. Sartenaer, Ph.L. Toint. Recursive trust-region methods for multiscale nonlinear optimization. *SIAM J. Optim.*, 2008.
- [5] J. Schulman et al. Trust Region Policy Optimization. *ICML*, 2015.
- [6] P.-A. Absil, R. Mahony, R. Sepulchre. *Optimization Algorithms on Matrix Manifolds*. Princeton, 2008.
- [7] Yu. Nesterov, B.T. Polyak. Cubic regularization of Newton method. *Math. Program.*, 2006.